FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Marcio Gabriel Schinor Mazega

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Quadro de entregas 3
4 Atividades realizadas 4
4.1 Atividade 1 — Agenda telefônica em Flask 4
4.2 Atividade 2 — Catálogo de filmes — Tom Hanks 5
4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação 8
4.4 Atividade 4 — Controle de acesso por papel — RBAC 12
4.5 Atividade 5 — Logs e auditoria 16
4.6 Atividade 6 — Upload e perfil de usuário 18
5 Atividades extras (opcional) 21
5.1 Atividade E1 — Documentação Swagger/OpenAPI 21
5.2 Atividade E2 — CI/CD com GitHub Actions 23
5.3 Atividade E3 — Observabilidade — health checks e métricas 25
6 Considerações finais 28
7 Declaração de autoria 28

1 Introdução

O presente relatório bimestral apresenta um resumo das atividades desenvolvidas na disciplina de Introdução à Computação em Nuvem (ISW055), entre 07/08/2026 e 02/10/2026. A disciplina teve como foco principal a construção e o reforço das habilidades de desenvolvimento da turma como um todo, ou seja, um compilado de desenvolvimento full-stack com um projeto prático de longo prazo, somado a boas práticas de infraestrutura e deploy em nuvem, além de segurança, pentest e microsserviços. O projeto proposto e tratado neste relatório é a plataforma de filmes do Tom Hanks, carinhosamente apelidada pelo autor deste trabalho de HANKS+: um catálogo com dados da API do TMDB que começou, na Atividade 2 (entregue em 20/08/2026), como uma aplicação única (backend em FastAPI e frontend em Angular) e terminou o bimestre como uma stack de sete containers publicada no Portainer, com serviços separados de autenticação, logs e armazenamento de objetos, além de monitoramento.

Com a disciplina, o autor espera aprender a desenvolver um sistema SaaS do início ao fim, com aprendizado reforçado em cibersegurança, engenharia de software, JWT, assinatura, cookies, entre outros. O relatório está organizado da seguinte forma: a Metodologia descreve as ferramentas, o ambiente e o critério usado para definir a data de cada entrega; o Quadro de entregas resume as atividades com as datas planejadas e realizadas; as seções Atividades realizadas e Atividades extras trazem uma ficha por atividade, com o que foi feito, as evidências e as dificuldades; e as Considerações finais encerram o relatório.

2 Metodologia

Todas as atividades foram desenvolvidas no mesmo repositório público do GitHub, como continuação de um único projeto (catálogo de filmes). A data e a hora de cada entrega foram extraídas do histórico de commits (`git log`) e conferidas na página do commit no GitHub. Os prints mostram as páginas dos commits de entrega no GitHub e o sistema em execução.

A stack do projeto foi Python e Angular. O backend foi desenvolvido em Python 3.12 com FastAPI, usando SQLAlchemy como ORM e Alembic para as migrações do banco, um MariaDB (compatível com MySQL) disponibilizado pela disciplina; o frontend foi feito em Angular 21 com TypeScript e é servido pelo mesmo container do catálogo. Ao longo do bimestre entraram o Redis (com Redis Streams, para a auditoria), o Garage como object storage compatível com S3 e o Prometheus e o Grafana para as métricas, além de dois serviços externos: a API do TMDB, para os dados dos filmes, e o Mailtrap, para o envio de e-mails em ambiente de teste. Cada serviço tem o próprio Dockerfile, e a stack completa é descrita em um único `docker-compose.yml`, que também permite executá-la localmente.

Para o deploy, foi usada uma infraestrutura previamente preparada pelo professor, baseada na plataforma Portainer: a stack é criada a partir do repositório no GitHub, as credenciais ficam como variáveis de ambiente da própria stack, e a aplicação é publicada no subdomínio `marcio-mazega-isw055.lapps.studio`, com o catálogo como único serviço de porta pública. Com o workflow do GitHub Actions criado em 19/09/2026 (atividade extra E2), cada push passou a rodar os testes automatizados (`pytest`) e uma stack de teste com verificação de saúde, publicando as imagens no GitHub Container Registry, que o Portainer passou a usar.

A data e a hora de cada entrega correspondem à data do autor do commit em que os requisitos da atividade ficaram completos no código, no horário de Brasília (UTC−03:00); commits posteriores que só acrescentaram prints ou documentação são citados nas fichas, mas não definem a data. Cada ficha também informa quando a issue de entrega foi aberta no GitHub; na Atividade 4, em que a issue foi aberta depois do prazo, a data registrada é a da issue, com a situação “entregue com atraso”. Os prints vêm de três fontes,

indicadas na legenda de cada um junto com a URL da imagem no repositório: imagens já versionadas no README, com o commit em que entraram; capturas das páginas de commit no GitHub feitas com Playwright em 05/10/2026; e prints tirados pelo autor. Nenhuma imagem foi editada além de ocultar dados sensíveis, como senhas, tokens e e-mails. A Atividade 1, feita em sala antes da criação do repositório, não tem commit associado: a ficha registra a entrega em sala e apresenta à parte, identificada como tal, uma reconstrução feita em outubro.

Ferramentas de IA, em especial o Claude Code, foram usadas como apoio no desenvolvimento, como mostra o roteiro de implementação da Atividade 6 versionado no repositório (`ai.md`), e na organização deste relatório; as decisões, os testes e a validação das entregas foram feitos pelo autor.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	07/08/2026 (em sala, horário não registrado)	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 20:27	entregue
3	Desacoplando o login — microsserviço de autenticação	Login, cadastro e esqueci--minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	28/08/2026 20:02	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	19/09/2026 16:50	entregue com atraso
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	19/09/2026 21:27	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	30/09/2026 09:59	entregue
E1	Documentação Swagger/ OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	22/09/2026 08:26	entregue
E2	CI/CD com GitHub Actions	Pipeline que testa e faz deploy a cada push.	sem prazo	19/09/2026 21:27	entregue
E3	Observabilidade — health checks e métricas	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.	sem prazo	19/09/2026 21:27	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	07/08/2026 (em sala, horário não registrado)
Situação	entregue
Evidência	Sem evidência da época: o código não foi versionado nem fotografado e foi perdido depois. Uma reconstrução feita em 05/10/2026 entra só como material complementar.
Link	—

O que foi feito. Durante a aula de 07/08/2026 construí a agenda telefônica proposta para nivelar a turma: uma única aplicação Flask com templates Jinja2, rotas para cadastrar e listar os contatos, com busca por nome, e persistência em um arquivo `contatos.json`, executada localmente com `flask run`.

Por que não há evidência. Não versionei o código em repositório nem fiz print do sistema rodando na época, e o código foi perdido quando formatei e reconfigurei o Windows e o WSL. Por isso registro a atividade sem evidência, em vez de apresentar um print produzido agora como se fosse daquela data.

Reconstrução (material complementar). Em 05/10/2026, durante a elaboração deste relatório, reescrevi a agenda para mostrar o tipo de sistema feito em sala: uma aplicação Flask com a rota `/`, que lista os contatos e filtra pelo nome com o parâmetro `busca`, e a rota `/cadastrar`, que recebe nome e telefone por formulário e grava tudo em `contatos.json`. **Não é o código original de 07/08/2026 e não comprova a data da entrega**; o código e o print abaixo são de outubro. O código está em um repositório separado, https://github.com/Schinor/Agenda-Telefonica-Allan, publicado no commit `be0f568` (05/10/2026 21:31).

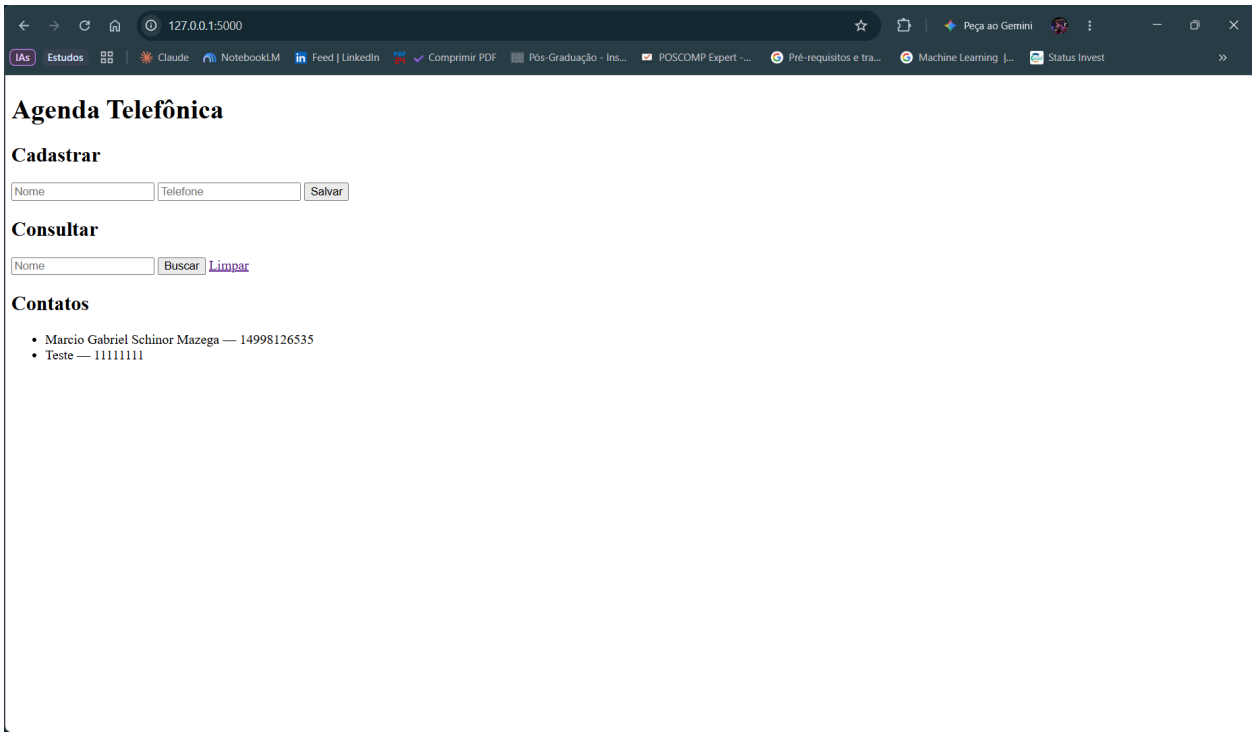


Figura 1: Atividade 1 — reconstrução da agenda telefônica em execução, feita em 05/10/2026; não é o sistema original de 07/08/2026 (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a1-agenda.png

Dificuldades e como foram resolvidas. Por ser uma atividade de nivelamento, não enfrentei muita dificuldade. Como Flask não é a minha stack, no começo estranhei um pouco justamente pela simplicidade do framework, mas depois o desenvolvimento fluiu muito bem.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 20:27
Situação	entregue
Evidência	GitHub — commit + README + print do catálogo
Link	https://github.com/Schinor/API-Filmes-Allan/commit/62ec4fce575b5b1dbc33787176835ba0c4a6eae8

O que foi feito. Construí o catálogo como uma aplicação com backend em FastAPI e frontend em Angular, publicada em `marcio-mazega-isw055.lapps.studio`. O backend busca o `person_id` de Tom Hanks em `/search/person` e a filmografia em `/person/{id}/movie_credits` na TMDB; pôster, título e sinopse vêm sempre ao vivo da API, e o pôster é exibido pela URL `image.tmdb.org/t/p/w500` sem baixar a imagem. A chave da TMDB fica só no servidor, em variável de ambiente.

Favoritos e comentários são gravados no MariaDB individual da disciplina, nas tabelas `usuarios`, `favoritos` e `comentarios`, com restrição única em `(usuario_id, tmdb_movie_id)` para impedir favoritar duas vezes o mesmo filme. A aplicação tem login e cadastro próprios, e na entrega todas as consultas, inclusões e exclusões de favoritos e comentários usavam o `usuario_id` de quem está logado, obtido do token, e não um identificador enviado pelo cliente. Mais tarde, em 19/09/2026 (commit `5cf318b`), a listagem

de comentários passou a ser pública, para que os comentários da comunidade e a moderação da atividade 4 funcionassem; os favoritos continuam isolados por usuário.

A entrega está no commit `62ec4fc` (20/08/2026 20:27), o último dos três commits daquele horário, que acrescenta o Dockerfile e o README com a menção ao professor. A issue de entrega #1 foi aberta às 20:32 do mesmo dia.

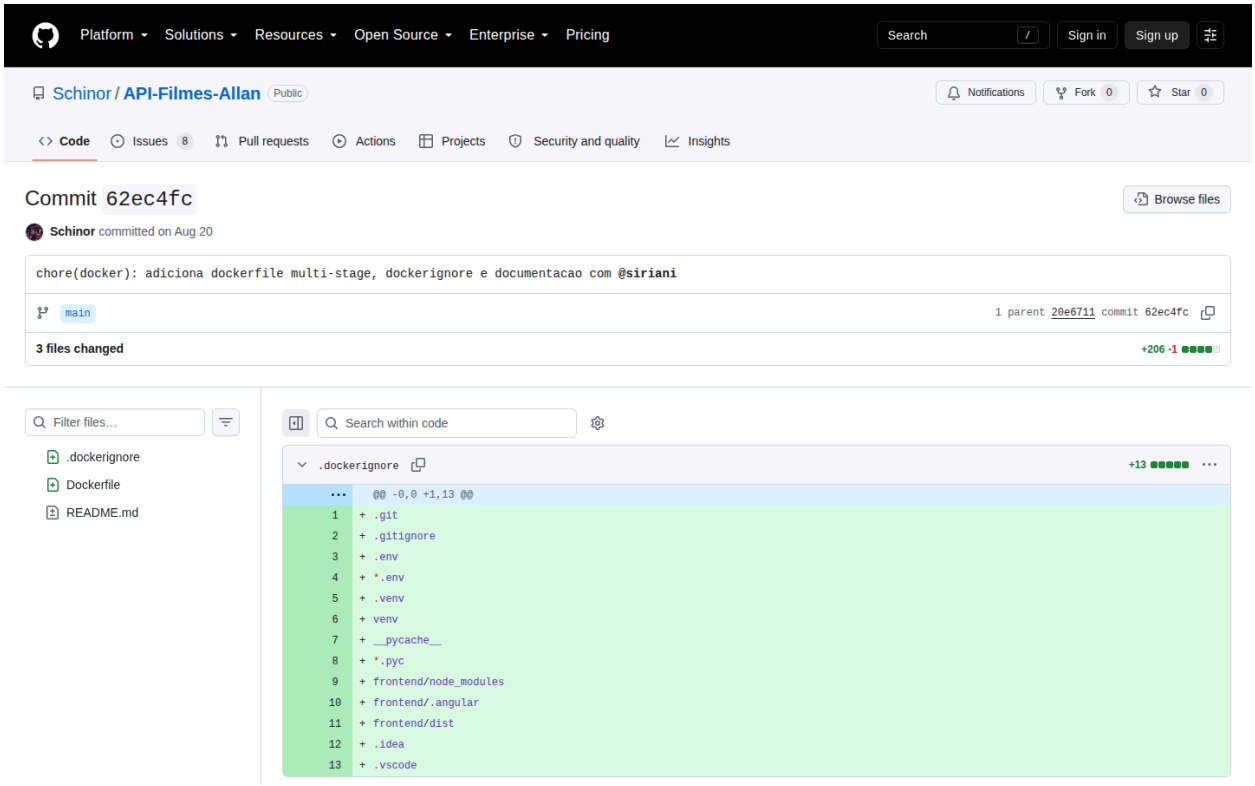


Figura 2: Atividade 2 — página do commit `62ec4fc` no GitHub, com o repositório, a mensagem e o hash. O GitHub exibe a data em formato relativo ("on Aug 20"); a data e hora exatas, 20/08/2026 20:27 (GMT-3), aparecem ao passar o mouse sobre ela e estão no link da ficha (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a2--commit.png

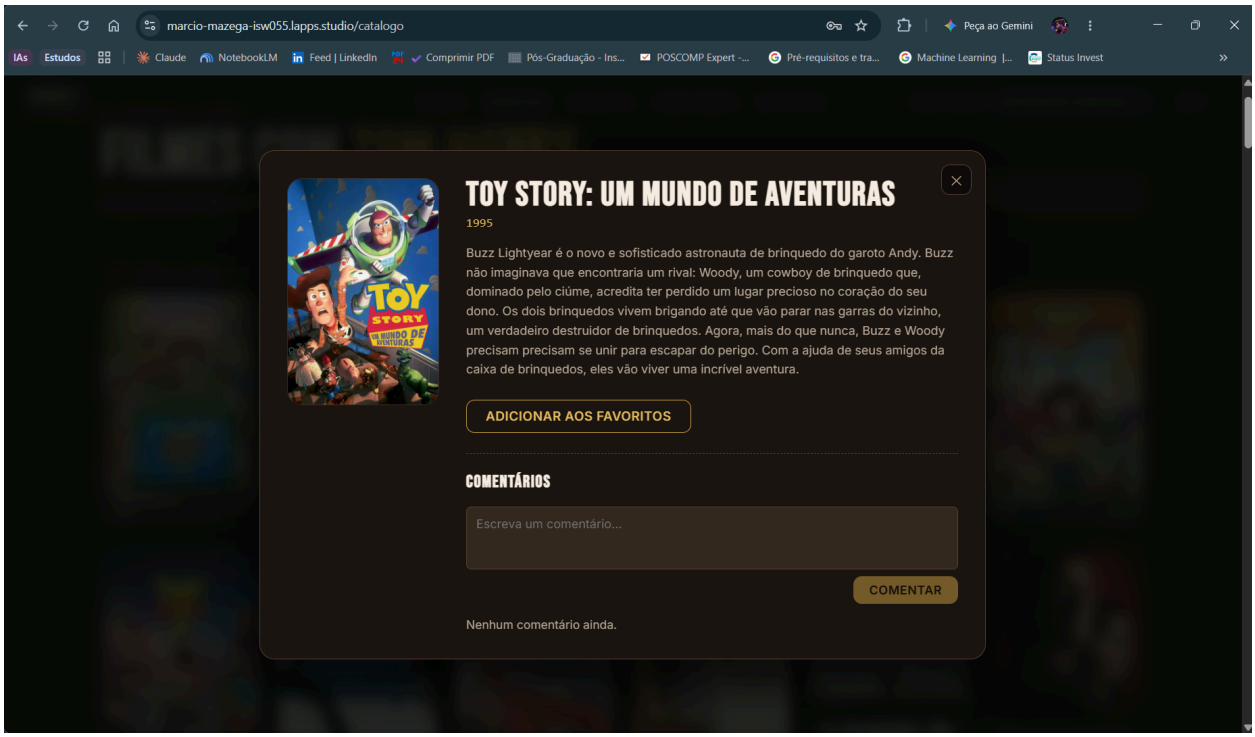


Figura 3: Atividade 2 — catálogo com pôster, título e sinopse vindos da TMDB (imagem do README, adicionada no commit `706c944` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SitePosterSinopse.png

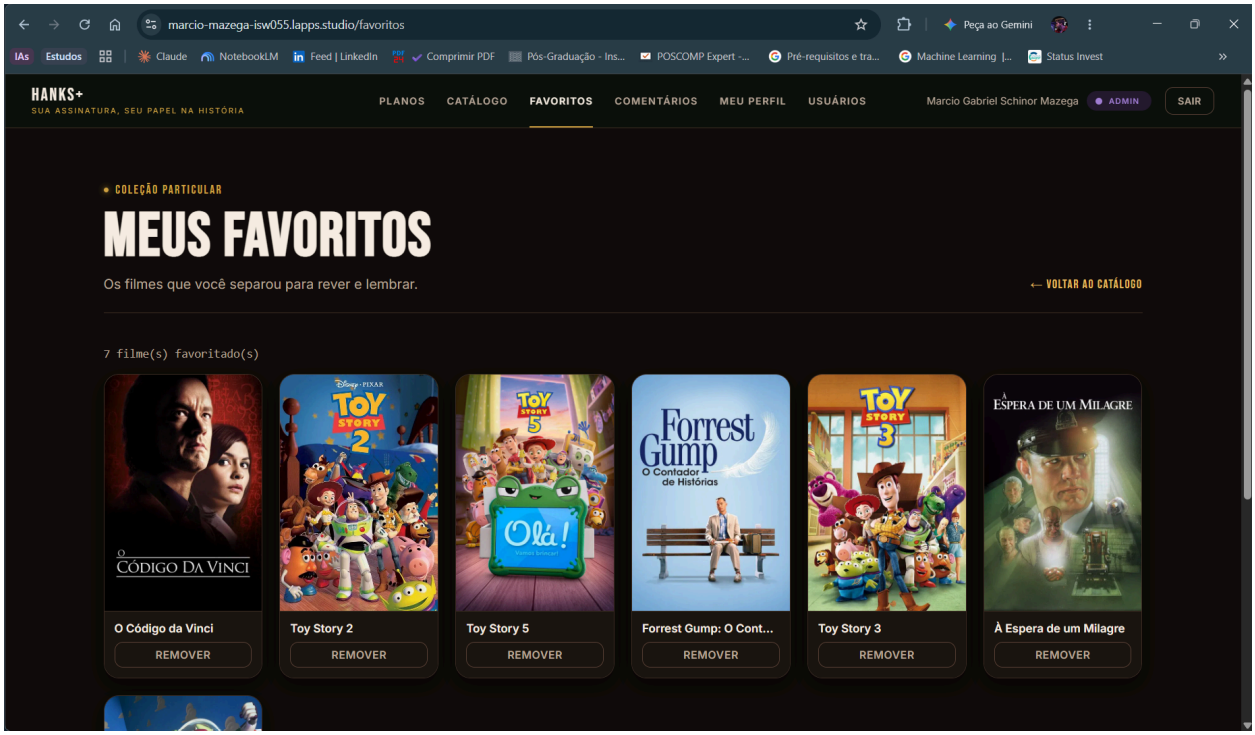


Figura 4: Atividade 2 — primeira conta com 7 filmes favoritados (imagem do README, adicionada no commit `706c944` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteAt2UserA.png

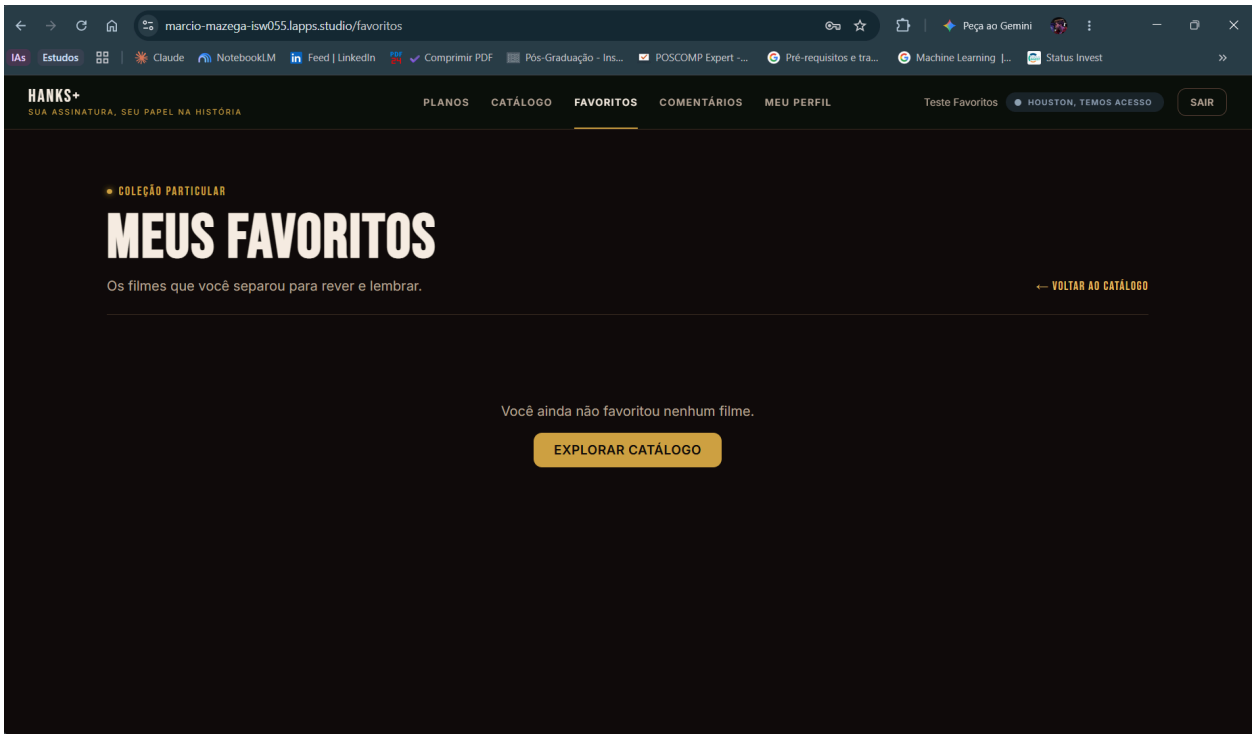


Figura 5: Atividade 2 — segunda conta, logada no mesmo endereço, sem nenhum favorito (imagem do README, adicionada no commit `706c944` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteAt2UserB.png

Dificuldades e como foram resolvidas. A maior dificuldade foi integrar a API do TMDB: tive problemas para montar a autenticação e inserir as credenciais da API, e para resolver pedi ajuda ao meu amigo Leonardo Ricci. A versão entregue aceita tanto a chave da API v3, enviada como parâmetro `api_key`, quanto o token de leitura v4, enviado no cabeçalho `Authorization: Bearer` (commit `7324dc4`). O outro problema foi o bcrypt: mesmo digitando a senha certa, a verificação retornava erro. Na entrega, o hash e a conferência da senha usam diretamente a biblioteca `bcrypt` (`hashpw` e `checkpw`), com a versão fixada em 4.2.0, no mesmo commit.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	28/08/2026 20:02
Situação	entregue
Evidência	GitHub — commit + docker-compose.yml + prints da recuperação de senha funcionando
Link	https://github.com/Schinor/API-Filmes-Allan/commit/4231b4cf19cabb760ac6c12accd950c8537a9640

O que foi feito. Extraí login, cadastro, papéis e recuperação de senha do catálogo para um segundo serviço, o `auth-service`, também em FastAPI. No `docker-compose.yml` ele não tem `ports` mapeado para o host: fica apenas na rede interna `filmes-network` e é chamado pelo catálogo pelo nome do serviço. O catálogo continua sendo o único ponto de entrada público.

A recuperação de senha usa a tabela `reset_tokens` (token aleatório e único, `usuario_id`, `criado_em`, `expira_em` igual a `criado_em` mais 30 minutos e `usado`). O link é enviado por SMTP pelo Mailtrap em ambiente de desenvolvimento e aponta para a rota `/reset-password` do catálogo, que repassa ao auth--service. Antes de trocar a senha, o serviço confere se o token existe, se ainda não expirou e se ainda não foi usado; qualquer falha recusa a troca, e depois da troca o token é marcado como usado. Na entrega, os papéis `usuario` e `admin` ficavam na coluna `role` da tabela `usuarios`.

O código ficou completo em 27/08/2026, entre 20:38 e 22:05 (serviço, migrações, envio de e-mail e a rota do link no catálogo). A data registrada é a do commit `4231b4c` (28/08/2026 20:02), que ajustou o `docker-compose.yml` para usar as imagens publicadas e nomes de container próprios, necessário para a stack subir no Portainer. A issue de entrega #2 foi aberta às 20:04 do mesmo dia.

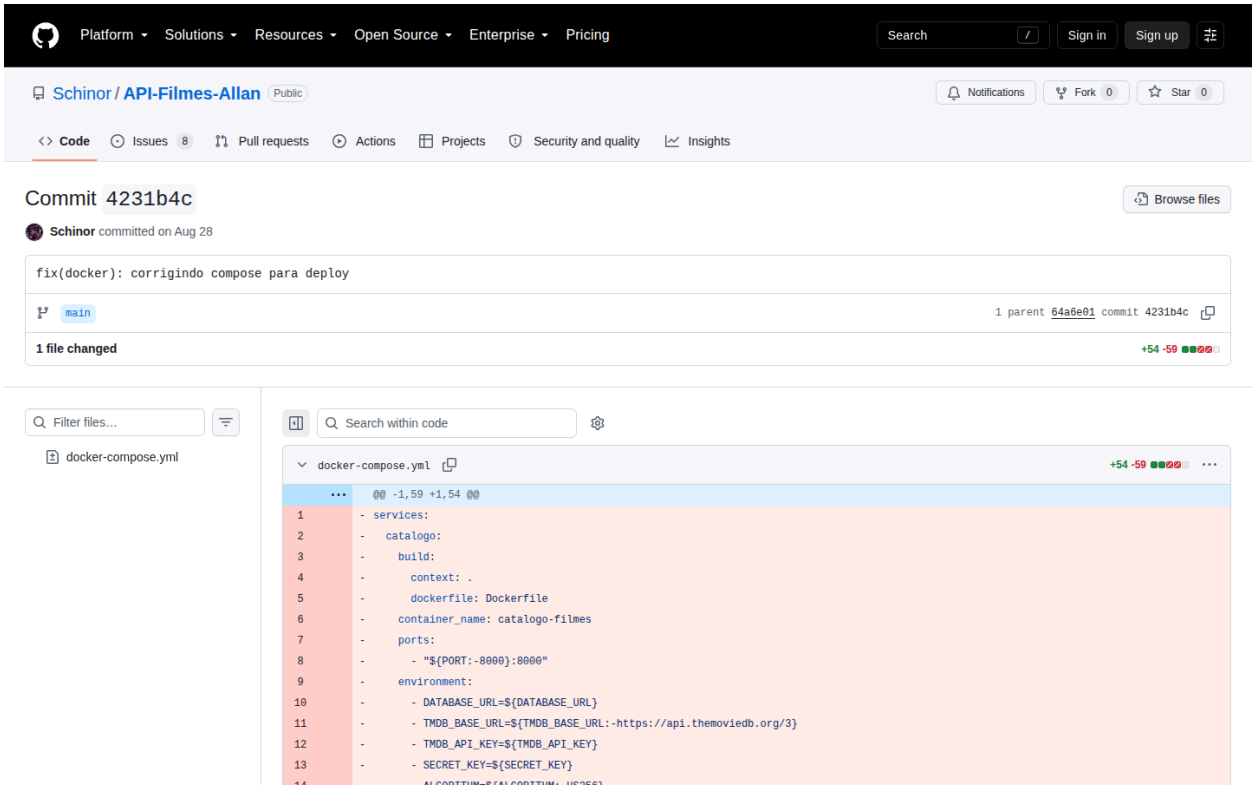


Figura 6: Atividade 3 — página do commit `4231b4c` no GitHub, com o repositório, a mensagem e o hash. O GitHub exibe a data em formato relativo ("on Aug 28"); a data e hora exatas, 28/08/2026 20:02 (GMT-3), aparecem ao passar o mouse sobre ela e estão no link da ficha (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a3--commit.png

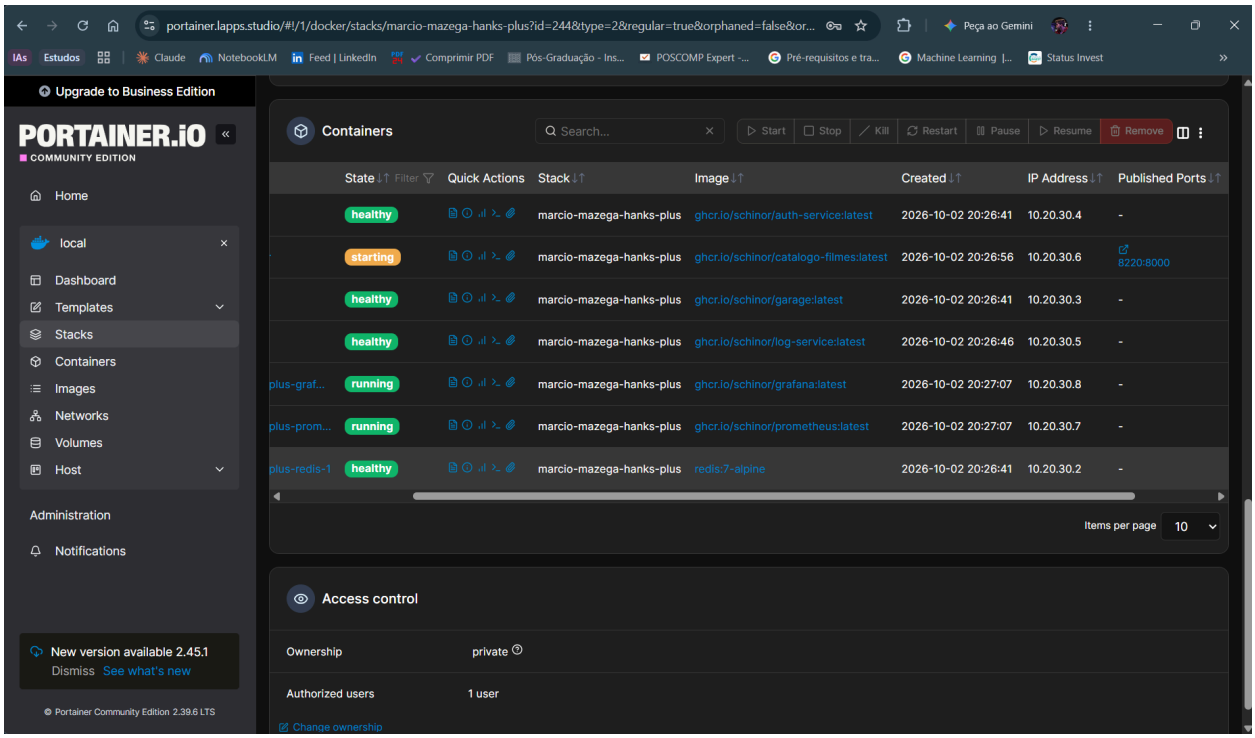


Figura 7: Atividade 3 — containers da stack no Portainer: só o catálogo publica porta; auth-service sem porta publicada (imagem do README, adicionada no commit `706c944` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SitePortainerContainers.png

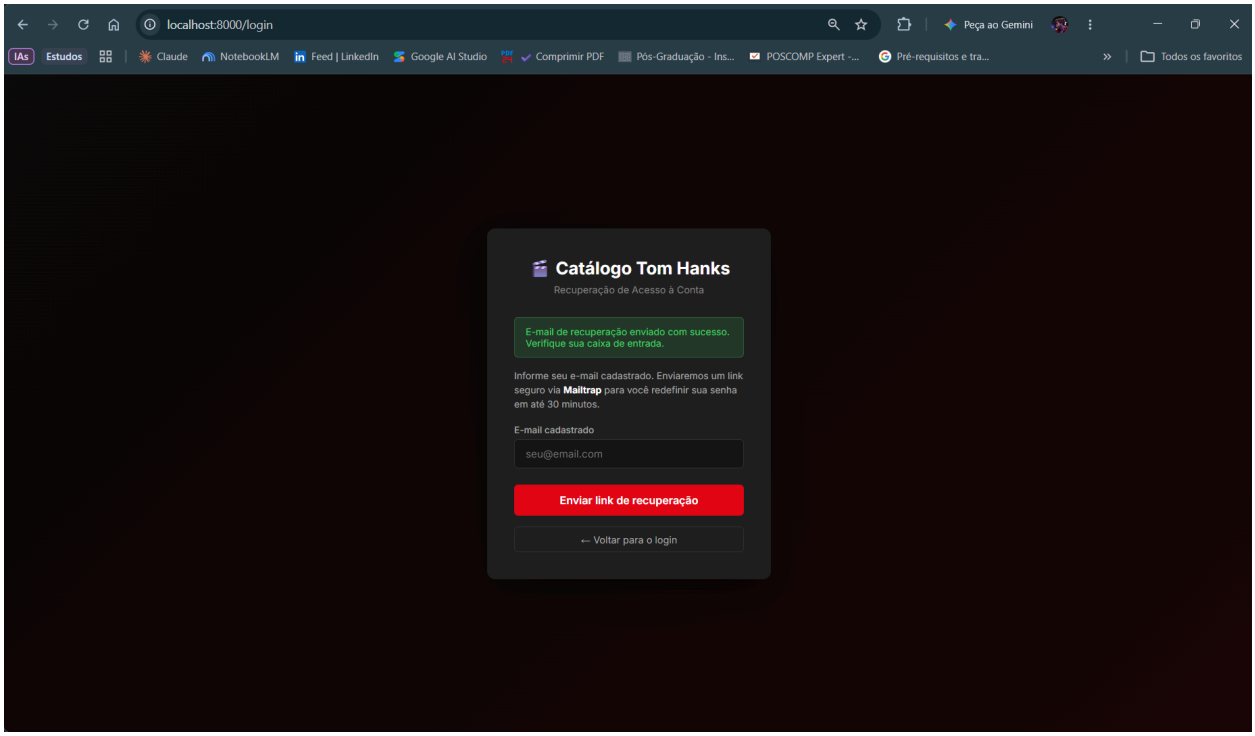


Figura 8: Atividade 3 — pedido de recuperação de senha na tela de login (imagem do README, adicionada no commit `64a6e01` em 27/08/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/Site_Funcionando.png

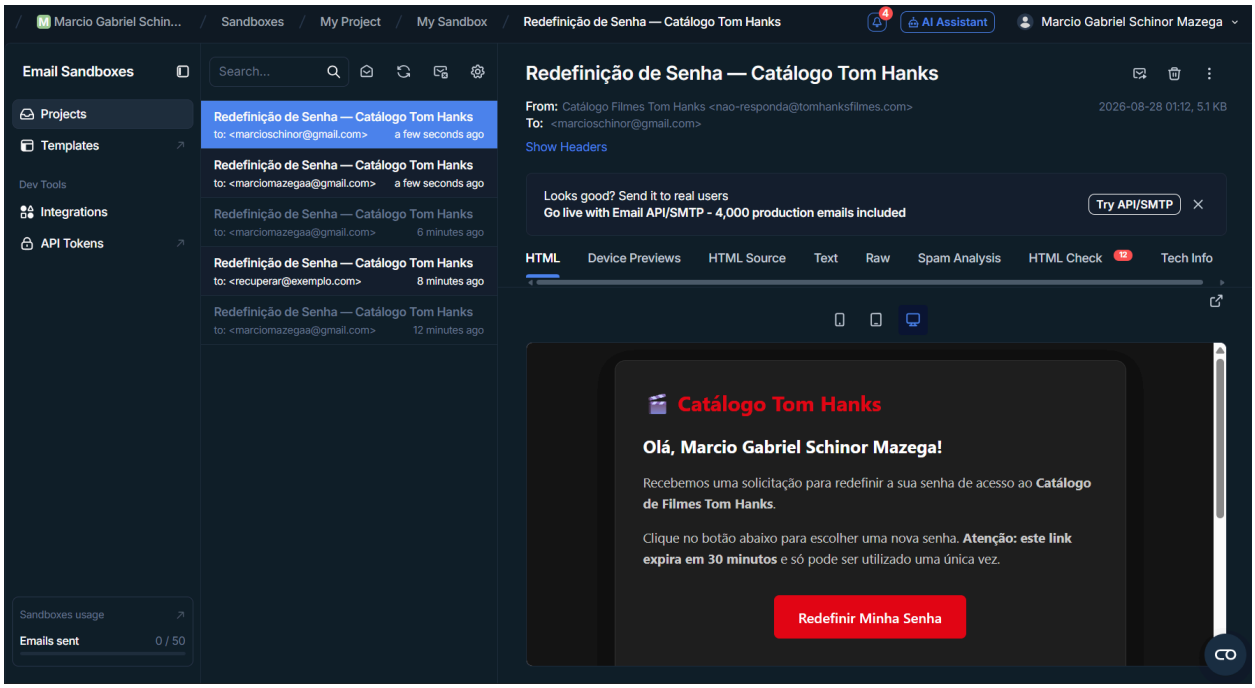


Figura 9: Atividade 3 — e-mail de redefinição recebido no Mailtrap (imagem do README, adicionada no commit `64a6e01` em 27/08/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/Site_Funcionando1.png

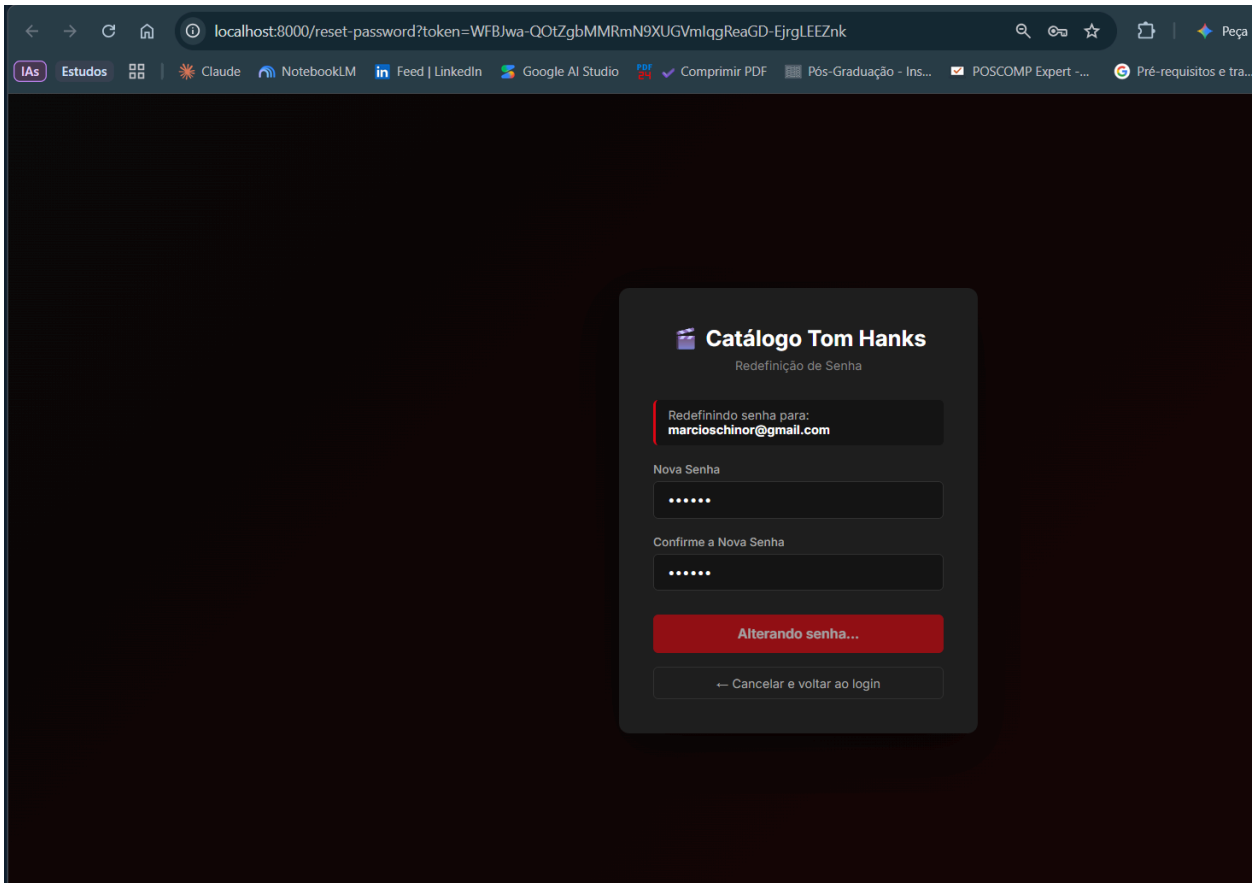


Figura 10: Atividade 3 — redefinição de senha pela rota do catálogo (imagem do README, adicionada no commit `64a6e01` em 27/08/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/Site_Funcionando2.png

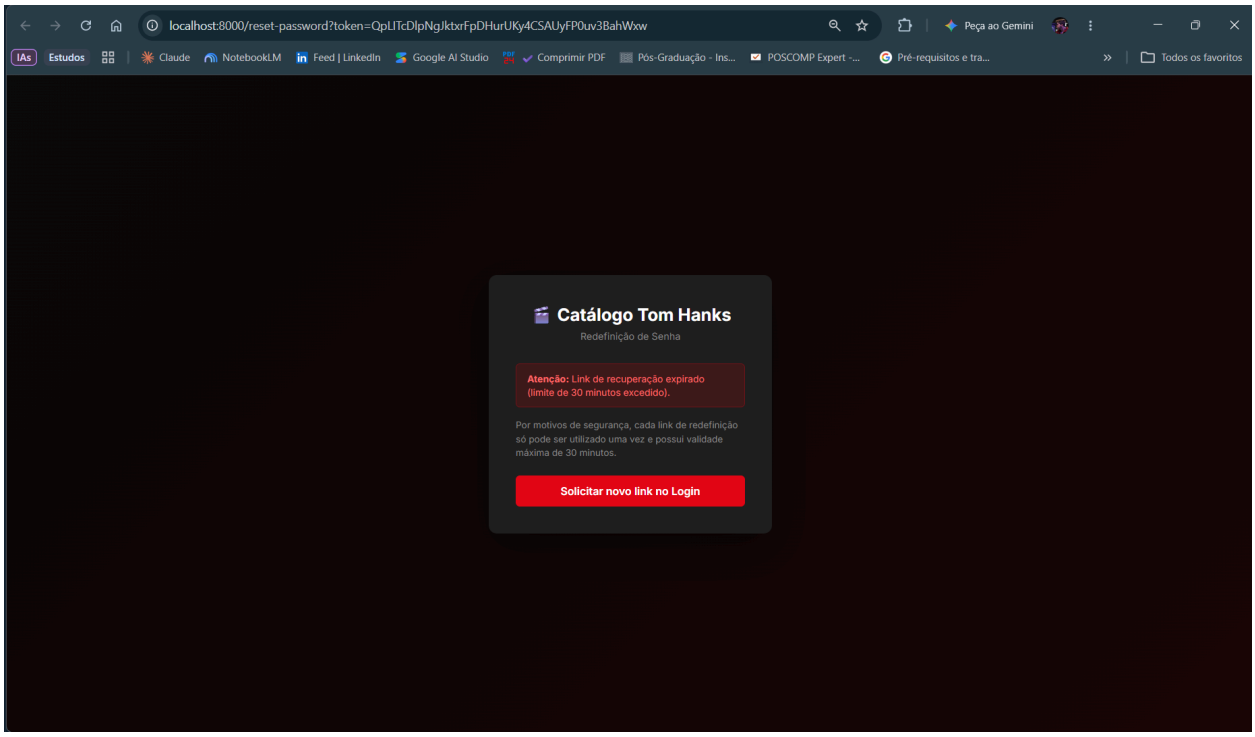


Figura 11: Atividade 3 — link expirado sendo recusado (imagem do README, adicionada no commit `64a6e01` em 27/08/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/Site_Funcionando3.png

Dificuldades e como foram resolvidas. A maior dificuldade foi configurar a plataforma do Mailtrap para o envio do e-mail de recuperação, e novamente contei com a ajuda do meu amigo Leonardo Ricci. O histórico mostra os ajustes daquela noite: um DNS público no `docker-compose.yml` e a limpeza de espaços e aspas nas credenciais do Mailtrap lidas do ambiente (`f9ec1ce`, 27/08 21:58), e um fallback de rotas no backend para que o link `/reset-password` do e-mail abrisse a tela do Angular (`478c45c`, 27/08 22:05). No dia seguinte, troquei `build:` por `image:` no `docker-compose.yml` para a stack subir no Portainer (`4231b4c`).

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	19/09/2026 16:50
Situação	entregue com atraso
Evidência	GitHub — commit + print do 403 e da ação de admin
Link	https://github.com/Schinor/API-Filmes-Allan/commit/9ab595372648014053f97bf12e1a362b4ba6de81

Sobre a data. Registro esta atividade como entregue com atraso. Uma primeira versão foi para o repositório em 04/09/2026 às 19:18 (commits `e66f1ff` e `fd5bf10`), dentro do prazo, mas a versão final e a issue de entrega #3 são de 19/09/2026: o último commit antes da issue é o `9ab5953` (16:43) e a issue foi aberta às 16:50.

O que foi feito. Na versão de 04/09, o catálogo consultava o auth-service a cada requisição protegida e decidia pela coluna `role`: apagar comentário de outro usuário (moderação) passou a ser exclusivo de

`admin`, e um usuário comum que tenta fazer isso recebe 403 ("Apenas o autor ou um administrador podem remover este comentário") mesmo chamando o endpoint diretamente, sem passar pela interface. O README documentou o que cada papel pode fazer e respondia Padrão A, o que batia com o código daquela data.

Em 13/09 (commits `14c0f83` e `90bad5e`) reestruturei o controle de acesso: as permissões passaram a ter nomes no formato `ação:recurso` (por exemplo `apagar:comentario-de-outro` e, depois, `visualizar:logs`), guardadas nas tabelas `permissions`, `roles` e `role_permissions` e definidas num único arquivo de seed; as rotas sensíveis passaram a usar a dependência `require_permission`; e o usuário comum foi dividido em quatro papéis (`amigo-do-wilson`, `preso-no-terminal`, `houston-temos-acesso` e `capitao-hanks`), além do `admin`. Em 19/09 bloqueei o autocadastro como administrador (`2f3d48f`), alinhei o README ao modelo novo (`db5bec3`) e adicionei os prints de moderação (`9ab5953`).

Hoje o sistema usa o Padrão B: no login, o auth-service grava `role` e `permissions` no JWT, e o catálogo valida a assinatura e lê as permissões localmente, sem chamada de rede a cada ação; só consulta o `/me` do auth-service quando o token não pode ser decodificado. Para voltar ao Padrão A, o catálogo deixaria de usar as permissões do token e consultaria o auth-service em cada requisição protegida: mudanças de papel teriam efeito imediato, mas cada ação ganharia uma chamada de rede e passaria a depender da disponibilidade do auth-service. No Padrão B, um token já emitido mantém as permissões antigas até expirar (24 horas).

Alterações visuais. No mesmo período a interface mudou bastante: em 04/09 criei uma landing page com os planos de usuário (`ad58424`), simplificada em 13/09 (`f2c280f`) junto com um guia de estilo do frontend (`cea50eb`); em 19/09 apliquei a identidade visual HANKS+, com a página de planos ligada aos papéis do RBAC e um design system documentado (`dc2337a`, `0c9dbc6` e `68e554e`); e em 28/09 entrou o logo SVG da plataforma (`17842d2`).

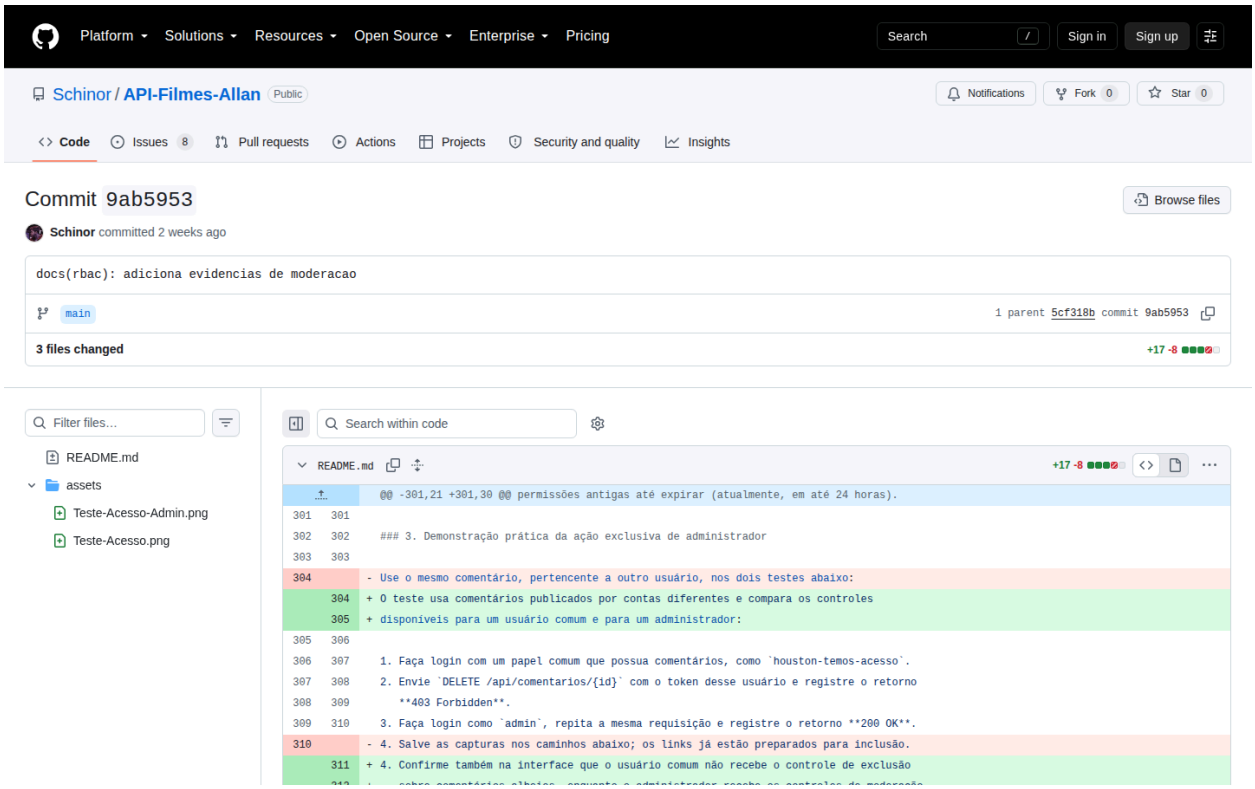


Figura 12: Atividade 4 — página do commit `9ab5953` no GitHub, com o repositório, a mensagem e o hash. O GitHub exibe a data em formato relativo (“2 weeks ago”); a data e hora exatas, 19/09/2026 16:43 (GMT-3), aparecem ao passar o mouse sobre ela e estão no link da ficha (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a4--commit.png

cole o print aqui
troque `arquivo: none` por `arquivo: "prints/nome.png"`

Figura 13: Atividade 4 — usuário comum recebendo 403 pelo Swagger ao tentar apagar o comentário de outro usuário. Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a4-403.png

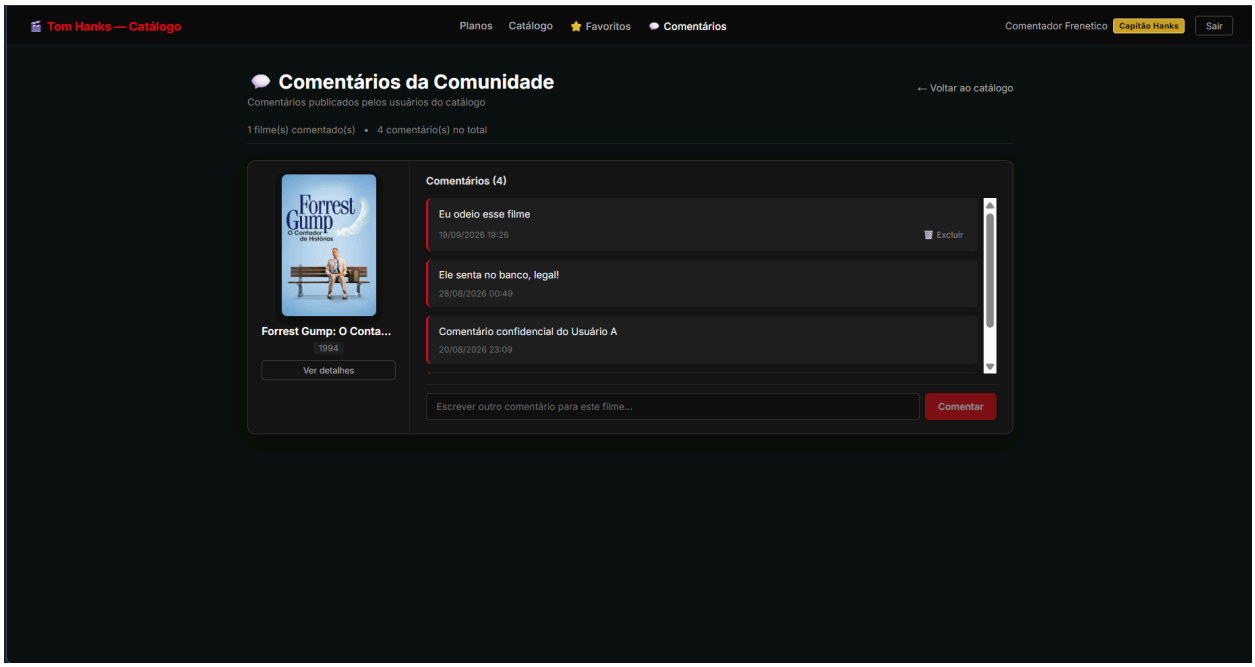


Figura 14: Atividade 4 — usuário comum sem o controle de exclusão nos comentários de outros usuários (imagem do README, adicionada no commit `9ab5953` em 19/09/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/Teste--Acesso.png

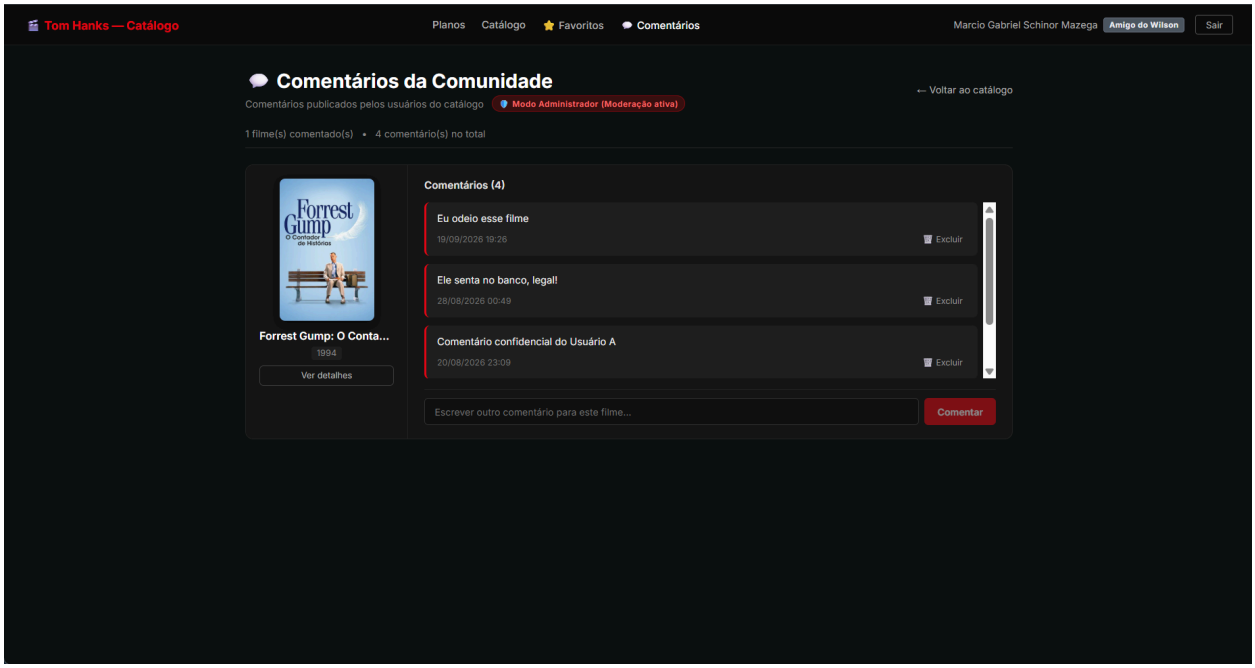


Figura 15: Atividade 4 — administrador com os controles de moderação (imagem do README, adicionada no commit `9ab5953` em 19/09/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/Teste-Acesso-Admin.png

Dificuldades e como foram resolvidas. Tive dois tipos de problema nesta atividade. O primeiro foi no banco: subi uma migração, apaguei o arquivo dela da minha máquina e depois tentei voltar a migração, o que gerou conflitos e outros erros entre o histórico de migrações e o estado real do banco. Resolvi sobrescrevendo o valor que estava no banco pelo que eu tinha no meu computador e, em seguida, rodando o `upgrade` do Alembic, que normalizou o banco de produção. Em 13/09 também centralizei as migrações no auth-service, removendo a pasta de migrações que não era mais usada no backend do catálogo (`f98641d`). O segundo foi de decisão arquitetural: demorei para decidir se criaria ou não uma estrutura de perfis (papéis e permissões em tabelas próprias) em vez de um simples campo `role`. Optei por criá-la porque

é mais organizada e mais plausível para o cenário do projeto, que deve virar um SaaS com planos de assinatura.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	19/09/2026 21:27
Situação	entregue
Evidência	GitHub — commit + print da consulta de logs pelo admin
Link	https://github.com/Schinor/API-Filmes-Allan/commit/a7a1f004dc0ee3b0ee30a60059e54f7da82cde34

O que foi feito. Criei o `log-service`, um terceiro serviço sem porta publicada, apoiado em um Redis também restrito à rede interna. O catálogo envia os eventos ao log-service autenticado por um token interno (cabeçalho `X-Internal-Token`), e o log-service grava cada evento em um Redis Stream com `XADD`; o auth-service não fala com o log-service, porque login e logout passam pelo catálogo, que registra esses eventos. Cada registro tem `usuario_id`, ação, timestamp, origem e o IP de quem fez a requisição. Se o log-service cair, a funcionalidade continua: o envio tem timeout curto e a falha vira só um aviso.

São registrados login, login com falha, logout, favoritar, comentar, apagar comentário e as tentativas negadas (`acesso_negado`). A consulta dos últimos eventos é a rota `/api/logs`, protegida pela permissão `visualizar:logs`, exclusiva de admin, reaproveitando o controle de acesso da atividade 4; um usuário comum que tenta consultá-la recebe 403, e essa tentativa também é registrada.

A entrega está no commit `a7a1f00` (19/09/2026 21:27), que coloca o log-service e o Redis no `docker-compose.yml`; o serviço e os eventos vêm dos commits `01337d5` e `6b87f5a`, do mesmo horário. A issue de entrega #4 foi aberta em 24/09/2026 às 19:35. Em 01/10/2026, revisando a atividade, percebi que os 403 devolvidos pelo auth-service nas rotas de administração repassadas pelo catálogo não geravam `acesso_negado`; corrigi com um tratamento central de 403 no catálogo (commit `00b23bd`), e desde então toda resposta 403 gera o evento.

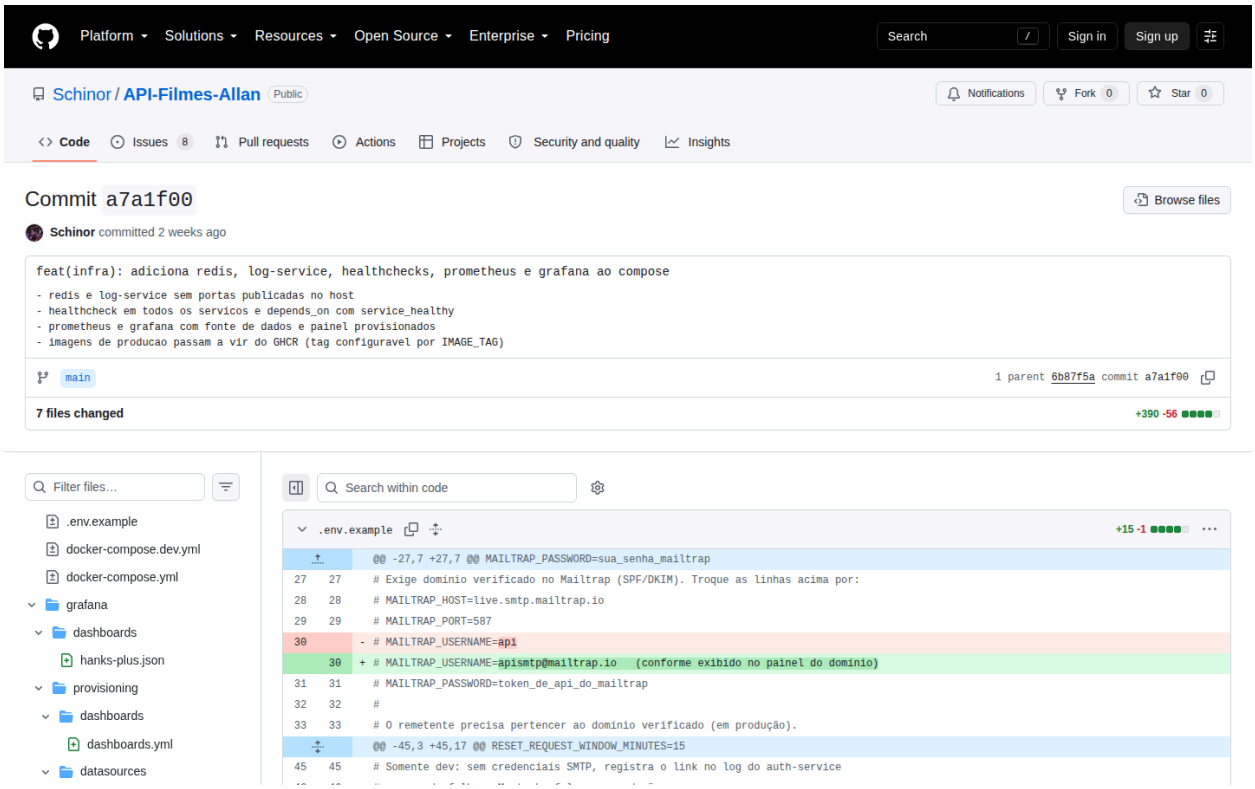


Figura 16: Atividade 5 — página do commit `a7a1f00` no GitHub, com o repositório, a mensagem e o hash. O GitHub exibe a data em formato relativo ("2 weeks ago"); a data e hora exatas, 19/09/2026 21:27 (GMT-3), aparecem ao passar o mouse sobre ela e estão no link da ficha (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a5--commit.png

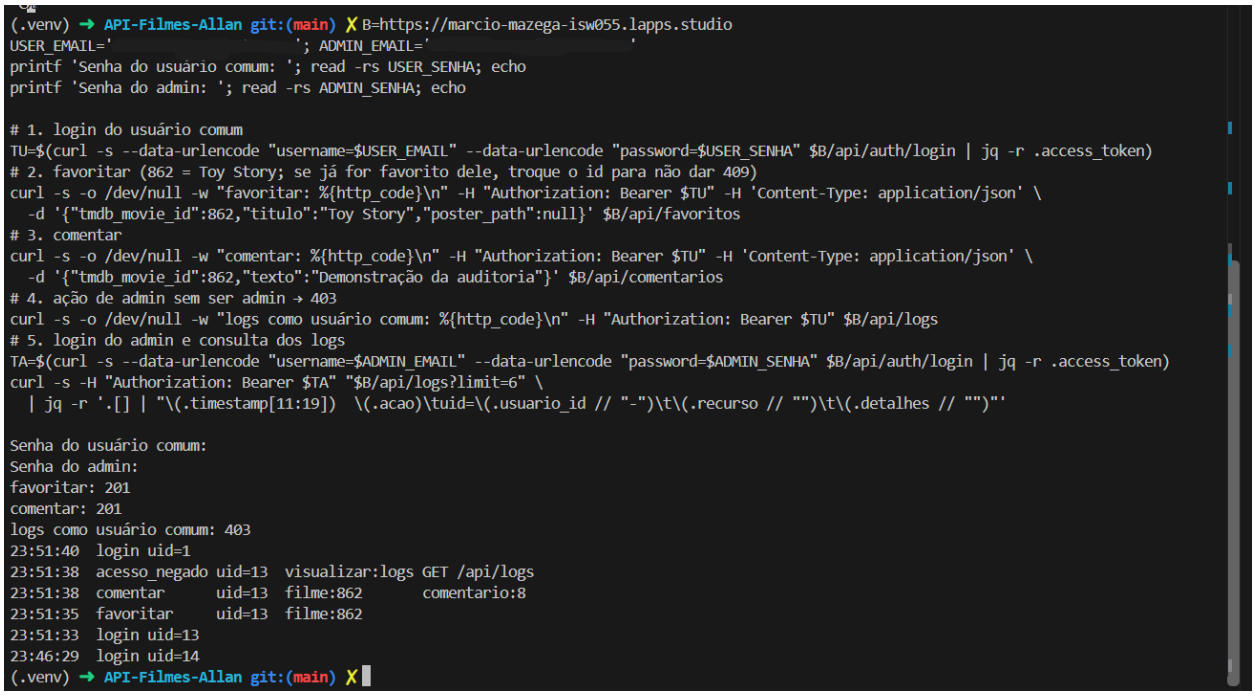


Figura 17: Atividade 5 — sequência login, favoritar, comentar, consulta negada (403) e consulta de logs como admin, com os eventos na ordem em que ocorreram (imagem do README, adicionada no commit `b867ada` em 02/10/2026, e-mails ocultados). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteLogsAdminScript.png

Dificuldades e como foram resolvidas. A stack não subia no Portainer. O servidor é compartilhado entre os alunos e os pools de rede padrão do Docker estavam esgotados ("all predefined address pools have been fully subnetted"); resolvi criando uma rede específica para a stack, com a sub-rede definida no

`docker-compose.yml`, para impedir que o Docker tentasse criar uma rede padrão (`ae3ce53`, 22/09/2026). No mesmo dia apareceu outro bloqueio: o Portainer não permite bind mount de caminho do host para usuários que não são administradores, então o Prometheus e o Grafana passaram a usar imagens próprias com a configuração embutida (`8c16172`).

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	30/09/2026 09:59
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/Schinor/API-Filmes-Allan/commit/ebfba41e7c57ef6c14db2e0aacd0c49d623b3a0a

O que foi feito. Implementei a página de perfil com nome, foto, bio e filmes favoritados. Como object storage usei o Garage no lugar do MinIO: o repositório da edição comunitária do MinIO foi arquivado em 2026, o Garage é gratuito, compatível com S3 e roda como um único container. Ele fica sem porta pública, com volumes próprios, e o banco guarda apenas a chave do objeto na tabela `perfis`, nunca o arquivo nem a URL.

O upload aceita no máximo 2 MB, lidos em partes (acima disso, 413), e confere o tipo pelo conteúdo do arquivo, não pela extensão: só JPEG, PNG e WEBP passam (o resto recebe 415). A imagem é regravada como WEBP 512×512, o que remove os metadados EXIF. Para exibir a foto, escolhi URLs pré-assinadas em vez de bucket público, servidas por um proxy `/storage` no catálogo; a URL é gerada a cada leitura do perfil e expira depois do tempo de download do arquivo numa conexão de 256 kbps mais 10 minutos de margem, com mínimo de 60 segundos e máximo de 1 hora (cerca de 10 minutos para um avatar comum). Quem confere a assinatura e a validade é o próprio Garage. Editar o perfil compara o `usuario_id` da rota com o do token, então tentar editar o perfil de outra pessoa resulta em 403, inclusive para admin, e cada tentativa negada gera o evento `acesso_negado` no log-service.

A entrega está no commit `ebfba41` (30/09/2026 09:59), que corrigiu a ponte `/storage`; o restante (Garage, perfil, testes, página no Angular, script de demonstração e README) foi para o repositório entre 09:40 e 09:53 do mesmo dia. A versão foi publicada no Portainer em 02/10/2026 e a issue de entrega #8 foi aberta às 21:26 desse dia.

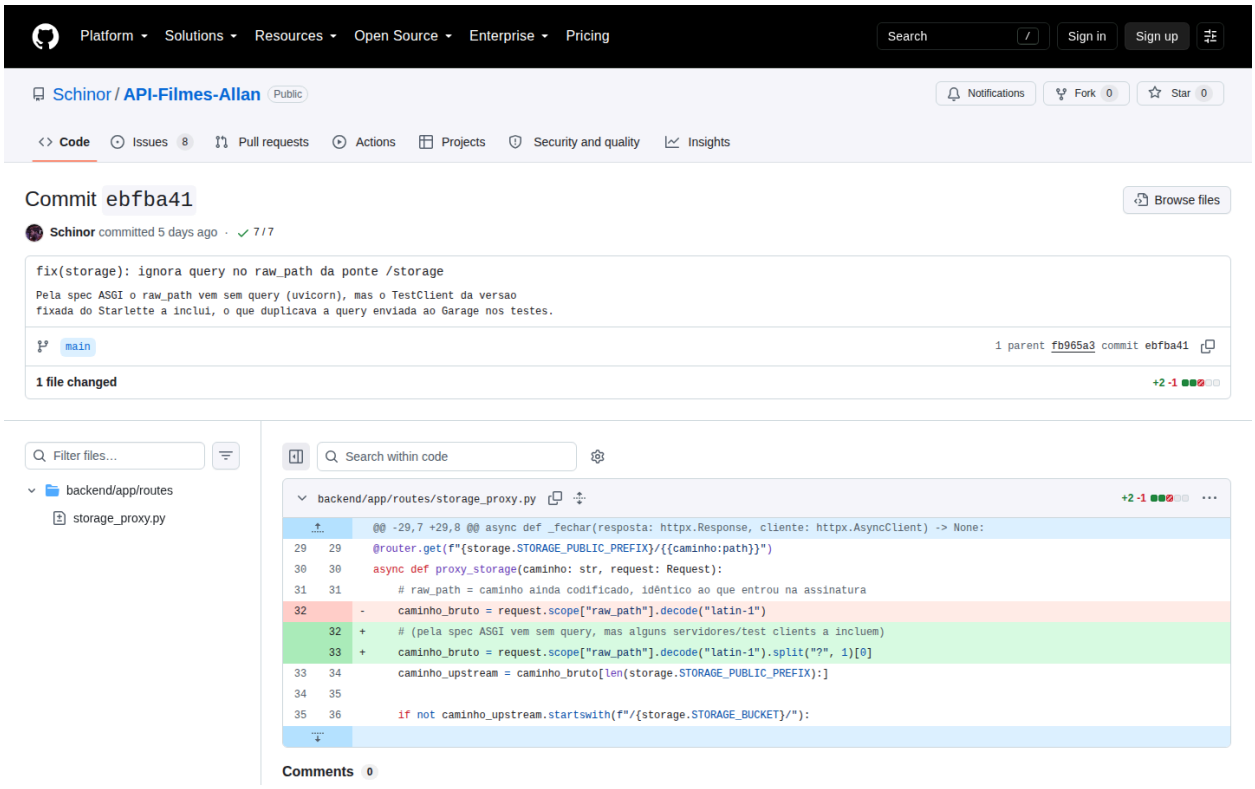


Figura 18: Atividade 6 — página do commit `ebfba41` no GitHub, com o repositório, a mensagem e o hash. O GitHub exibe a data em formato relativo ("5 days ago"); a data e hora exatas, 30/09/2026 09:59 (GMT-3), aparecem ao passar o mouse sobre ela e estão no link da ficha (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/a6--commit.png

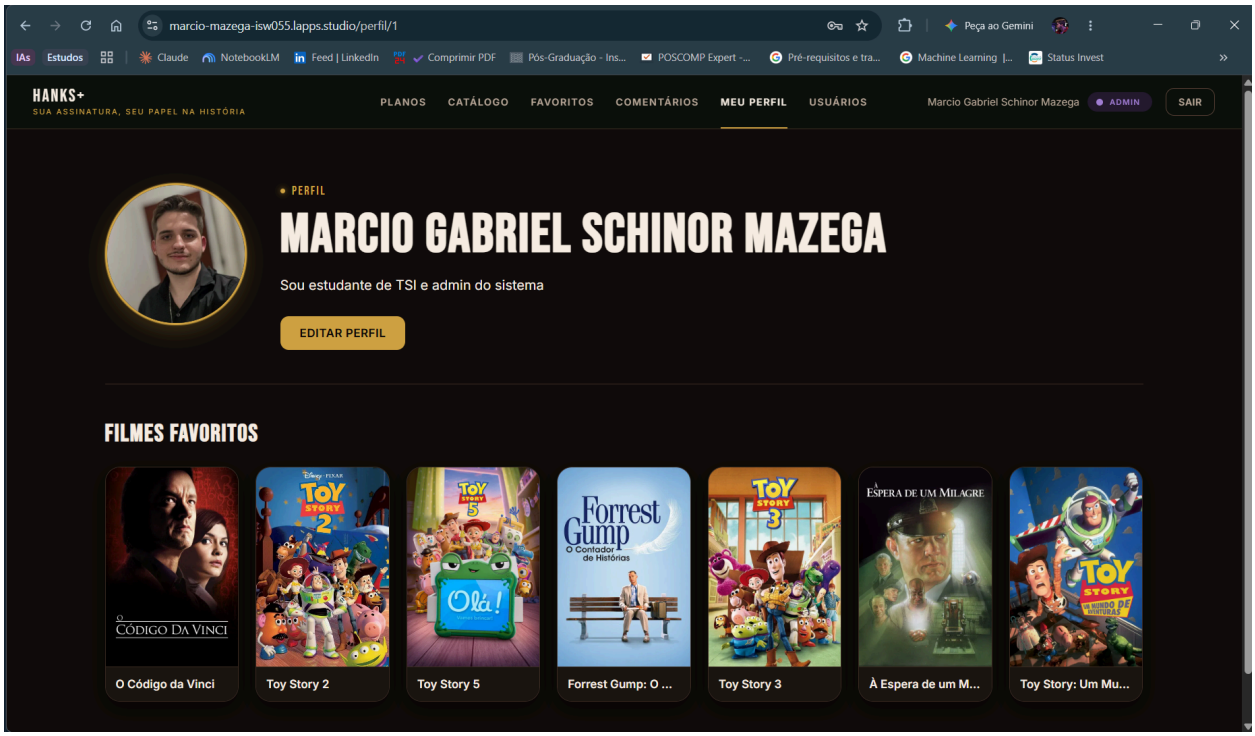


Figura 19: Atividade 6 — página de perfil com a foto enviada (imagem do README, adicionada no commit `1b0279d` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SitePerfilFoto.png

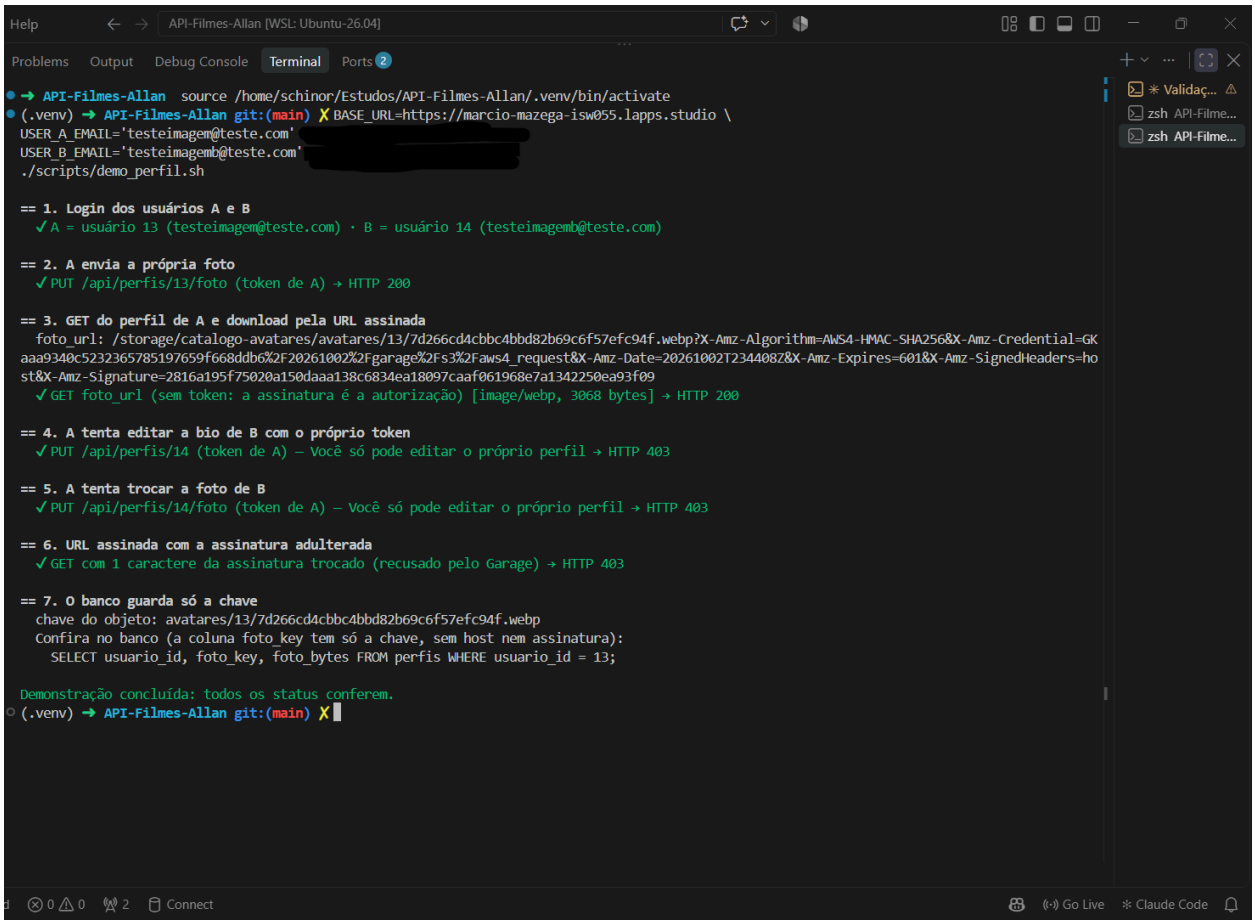


Figura 20: Atividade 6 — script de demonstração em produção: tentativas de editar a bio e a foto de outro usuário recusadas com 403 e URL adulterada recusada (imagem do README, adicionada no commit `9408d2d` em 02/10/2026, senhas ocultadas). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteGarageScript.png

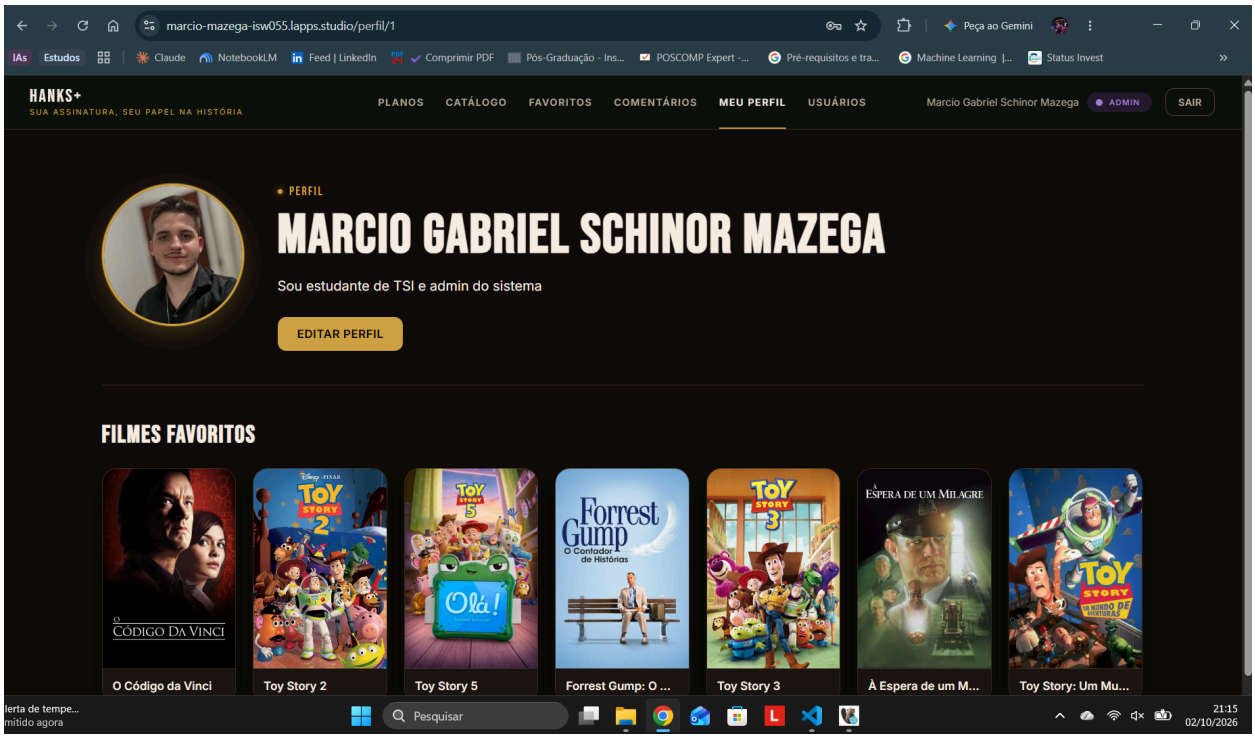


Figura 21: Atividade 6 — a foto continua no perfil depois de remover e recriar os containers (imagem do README, adicionada no commit `390c2b6` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SitePerfilPosDeletaroContainer.png

Dificuldades e como foram resolvidas. A principal decisão foi o object storage. Escolhi o Garage pela recomendação do professor e por ser gratuito, ao contrário do MinIO; também considerei o Swift (OpenStack), mas para guardar avatares seria como matar uma formiga com uma bazuca. Na implementação, os testes mostraram a query da URL assinada duplicada ao passar pela ponte `/storage`, porque o cliente de testes do Starlette inclui a query no `raw_path`; corrigi fazendo a ponte ignorar essa parte (`ebfba41`).

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	22/09/2026 08:26
Situação	entregue
Evidência	GitHub — commit + Swagger UI público do catálogo + print do “Try it out”
Link	https://marcio-mazega-isw055.lapps.studio/api/docs

O que foi feito. Documentei o catálogo e o auth-service. Como os dois são FastAPI, a especificação OpenAPI é gerada a partir do código: parâmetros com suas validações, corpos de requisição e resposta pelos schemas Pydantic e respostas de erro (400, 401, 403, 404, 409, 502, 503) declaradas em cada rota a partir de um módulo central, `core/api_responses.py`. O Swagger UI do catálogo é público em `/api/docs`, com Redoc em `/api/redoc`; o do auth-service só é acessível pela rede interna, coerente com a decisão da atividade 3, e essa escolha está justificada no README (a especificação do auth-service não foi exportada

para o repositório). As rotas de perfil da atividade 6 (`/api/perfis/*`) também aparecem na documentação publicada, com as respostas de erro do upload.

A entrega está no commit `bdce733` (22/09/2026 08:26); o print do "Try it out" entrou no README no commit `2fbd07e` (22/09/2026 21:02) e o link de produção do Swagger em `16d136c` (01/10/2026). A issue de entrega #6 foi aberta em 24/09/2026 às 19:41.

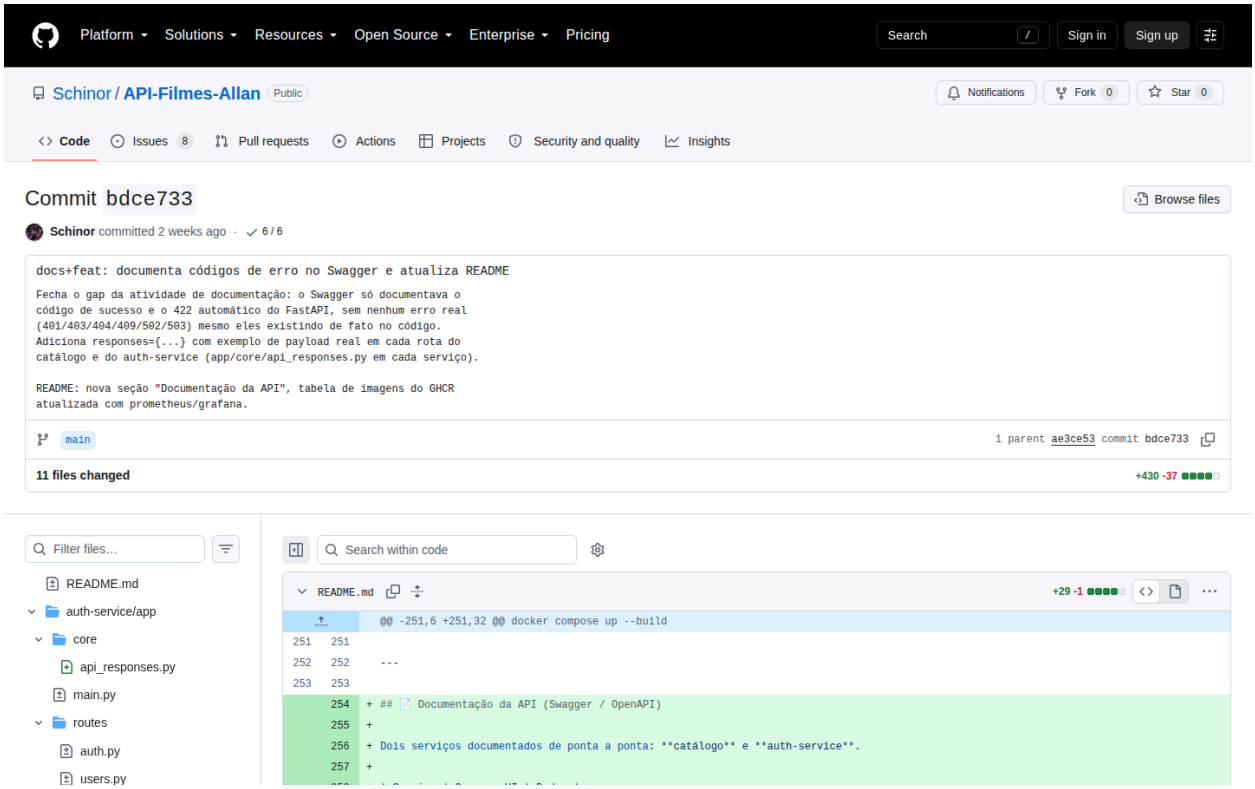


Figura 22: Atividade E1 — página do commit `bdce733` no GitHub, com o repositório, a mensagem e o hash. O GitHub exibe a data em formato relativo ("2 weeks ago"); a data e hora exatas, 22/09/2026 08:26 (GMT-3), aparecem ao passar o mouse sobre ela e estão no link da ficha (capturado em 05/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/docs/p1/prints/e1-commit.png

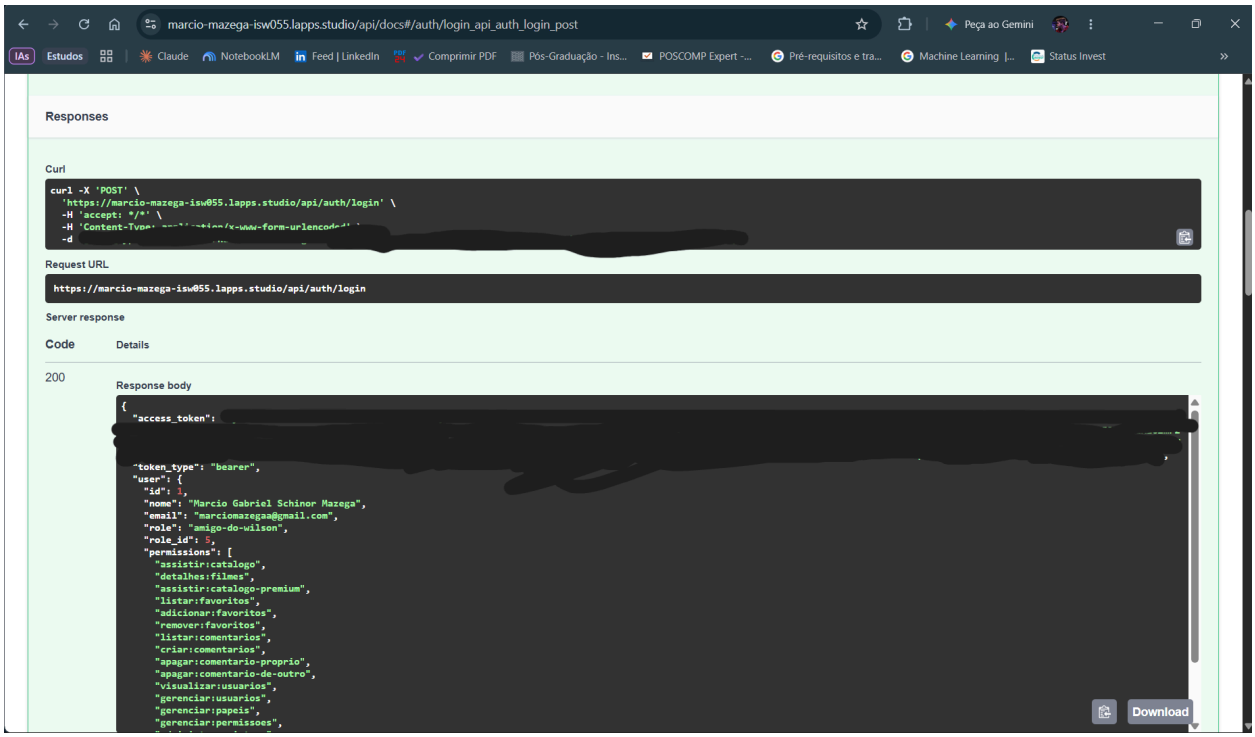


Figura 23: Atividade E1 — Swagger UI com `POST /api/auth/login` executado via "Try it out" em produção (imagem do README, adicionada no commit `2fbd07e` em 22/09/2026, senha e token ocultados). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteSwaggerUI.png

Dificuldades e como foram resolvidas. Eu achava que o FastAPI já documentava toda a API sozinho, tanto que cheguei a considerar a tarefa entregue. Revalidando, descobri que ele não entregava o que eu precisava: o Swagger só mostrava a resposta de sucesso e o 422 de validação gerado automaticamente, sem os erros que as rotas devolvem de fato (401, 403, 404, 409, 502 e 503). Resolvi declarando essas respostas em cada rota, com exemplos de payload, a partir do módulo central `core/api_responses.py` de cada serviço (`bdce733`).

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline que testa e faz deploy a cada push.
Data planejada	sem prazo
Data realizada	19/09/2026 21:27
Situação	entregue
Evidência	GitHub Actions — execução verde + imagem com tag do commit no GHCR
Link	https://github.com/Schinor/API-Filmes-Allan/actions/runs/37078516642

O que foi feito. Criei um workflow em `.github/workflows/` que, a cada push, roda os testes automatizados (`pytest`), sobe a stack de teste, espera os health checks e faz chamadas de verificação; se algo falha, o pipeline quebra. Quando o teste passa em push para `main`, as imagens são publicadas no GitHub Container Registry com duas tags, `latest` e `sha-` seguida do hash do commit. Os segredos do CI são gerados em tempo de execução e o login no registry usa o token automático do próprio workflow; em produção as credenciais ficam como variáveis de ambiente da stack no Portainer, criada a partir do repositório com atualização automática por consulta periódica (a cada 5 minutos). Como a opção de baixar de novo a

imagem a cada atualização é recurso pago do Portainer, a troca de versão é feita definindo `IMAGE_TAG` com a tag `sha-` desejada e executando "Pull and redeploy"; esse passo manual está documentado no README.

O workflow foi criado no commit `d521e62` (19/09/2026 21:27) e a primeira execução verde foi em 21/09/2026 às 19:38, quando os commits daquele dia foram enviados ao GitHub. O link da ficha é a execução verde do commit `706c944`, que publicou a tag `sha-706c944` usada pelos containers do print abaixo. A issue de entrega #5 foi aberta em 24/09/2026 às 19:40.

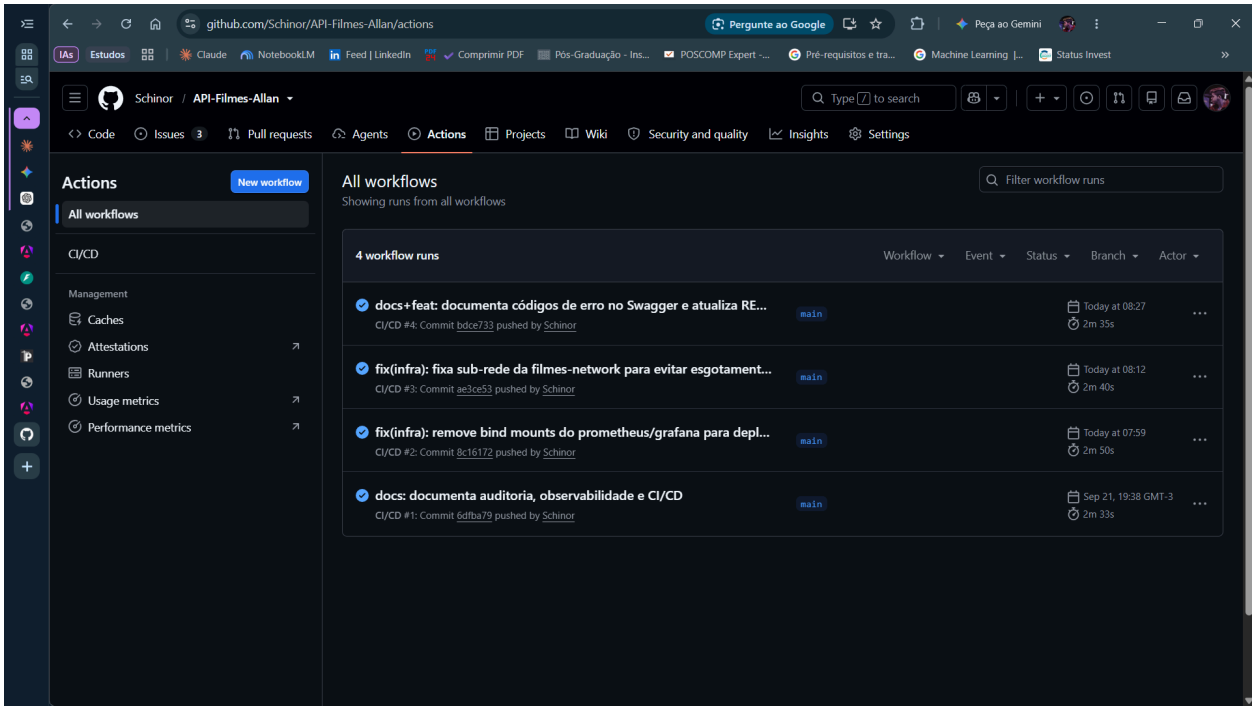


Figura 24: Atividade E2 — execuções verdes do workflow na aba Actions (imagem do README, adicionada no commit `ffad32e` em 22/09/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteCICD.png

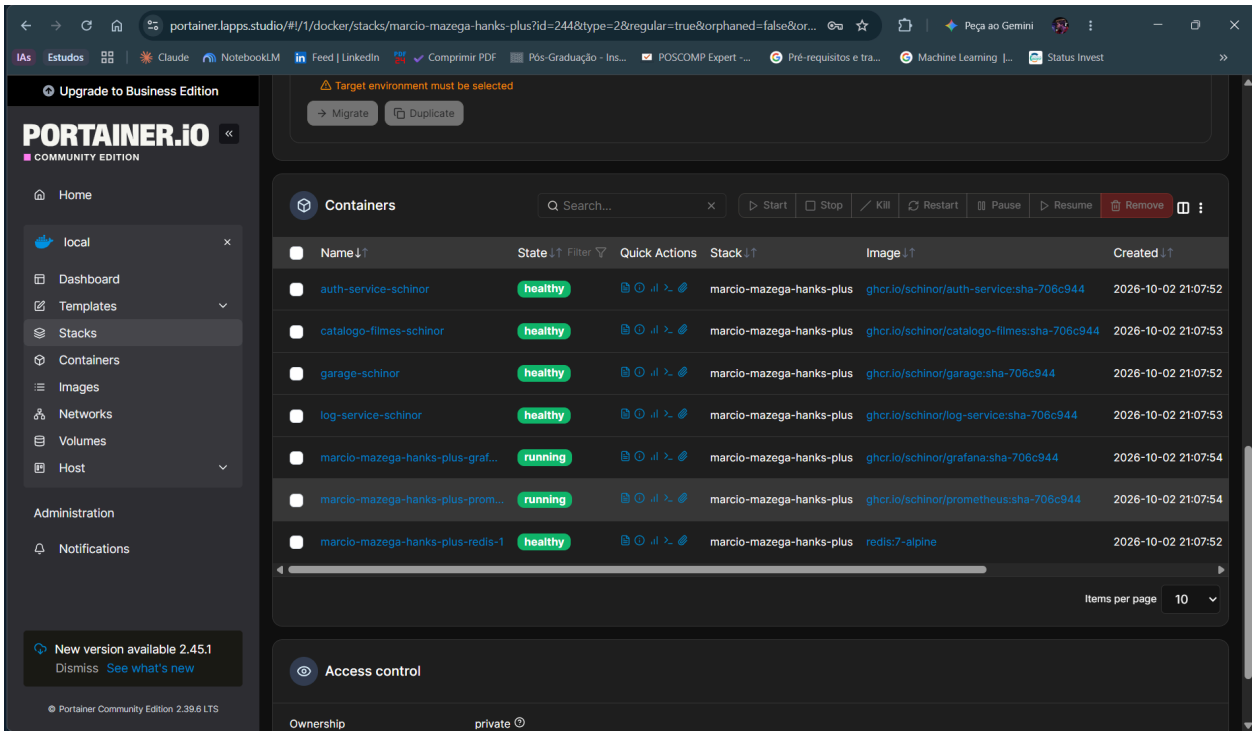


Figura 25: Atividade E2 — containers em produção rodando as imagens com a tag `sha-706c944`, e não a `latest` (imagem do README, adicionada no commit `b867ada` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SitePortainerComContainers.png

Dificuldades e como foram resolvidas. A parte mais complexa foi entender o fluxo de testes e a documentação do CI/CD: em que ordem rodar os testes, subir a stack de teste, esperar os health checks e só então publicar as imagens. Depois que entendi esse fluxo, a implementação foi bem tranquila.

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.
Data planejada	sem prazo
Data realizada	19/09/2026 21:27
Situação	entregue
Evidência	GitHub — commit + prints do docker ps nos dois estados + painel Grafana
Link	https://github.com/Schinor/API-Filmes-Allan/commit/a7a1f004dc0ee3b0ee30a60059e54f7da82cde34

O que foi feito. Cada serviço expõe `/health` com verificação de prontidão real e retorna 503 quando alguma dependência está fora: o catálogo testa o MariaDB, o auth-service e o Garage; o auth-service testa o MariaDB; o log-service testa o Redis. O `HEALTHCHECK` do Docker chama esse endpoint com Python, já que as imagens slim não trazem `curl` (o Garage usa o próprio binário), de modo que o `docker ps` mostra o estado de cada container. As métricas de requisição por rota, código de status e latência são expostas em `/metrics` com `prometheus-fastapi-instrumentator`, coletadas por um Prometheus e exibidas em um painel do Grafana, ambos na mesma stack.

O Grafana e o Prometheus não têm porta pública: o catálogo os expõe em `/grafana` e `/prometheus` e encaminha as requisições pela rede interna. O acesso não reaproveita a sessão (cookie) do usuário admin da aplicação; cada ferramenta tem a própria autenticação. O Grafana usa o login nativo dele, com o usuário

`admin` e a senha definida na variável `GRAFANA_ADMIN_PASSWORD` da stack. Como o Prometheus não tem login próprio, o proxy do catálogo exige HTTP Basic (usuário `admin` e senha `PROMETHEUS_PROXY_PASSWORD`), compara as credenciais em tempo constante (`secrets.compare_digest`), devolve 401 para credenciais erradas e responde 503 se a senha não estiver configurada, para nunca ficar aberto por esquecimento.

A entrega está no commit `a7a1f00` (19/09/2026 21:27), que colocou os health checks, o `/metrics`, o Prometheus e o Grafana na stack. Essa entrega foi parcial: o `/health` do catálogo testava só o MariaDB, e não as outras dependências. Revisando a atividade em 01/10/2026, identifiquei o que faltava e completei a verificação com o auth-service e o Garage no commit `00b23bd` (01/10/2026 10:14). A issue de entrega #7 foi aberta em 02/10/2026 às 21:25.

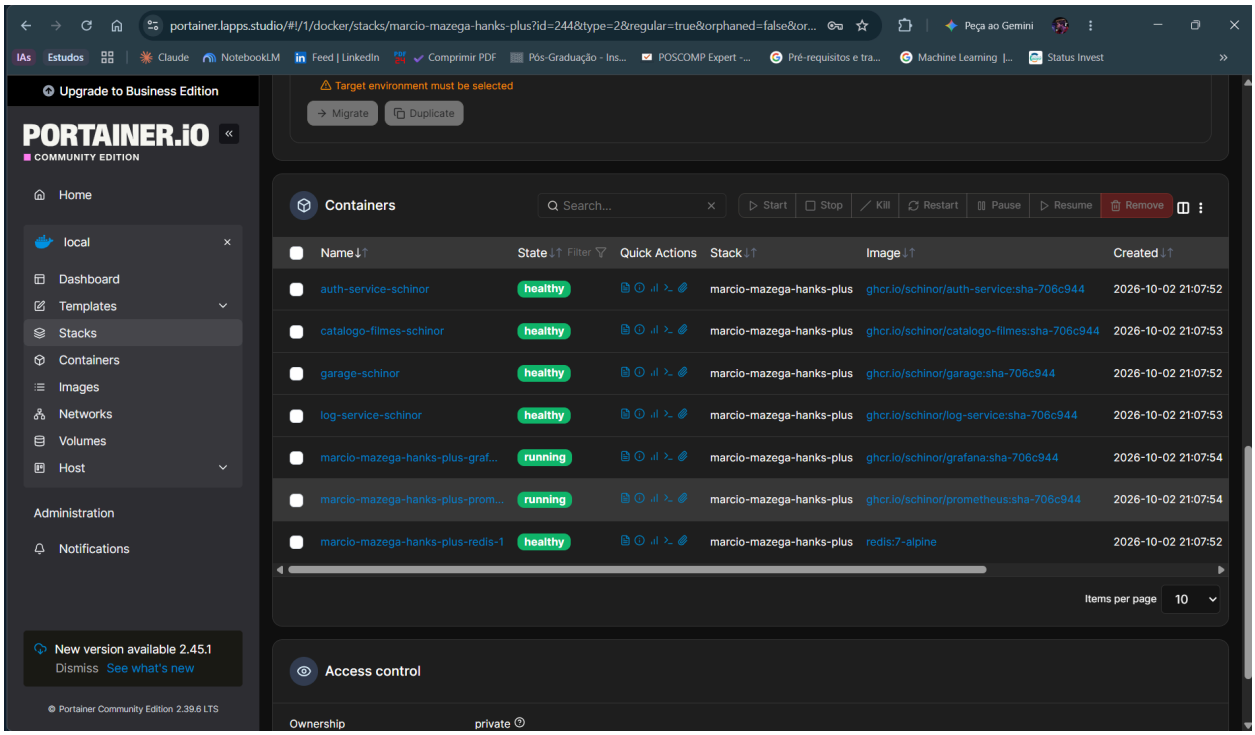


Figura 26: Atividade E3 — containers da stack com todos os serviços healthy; Grafana e Prometheus aparecem running por não terem healthcheck (imagem do README, adicionada no commit `b867ada` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SitePortainerComContainers.png

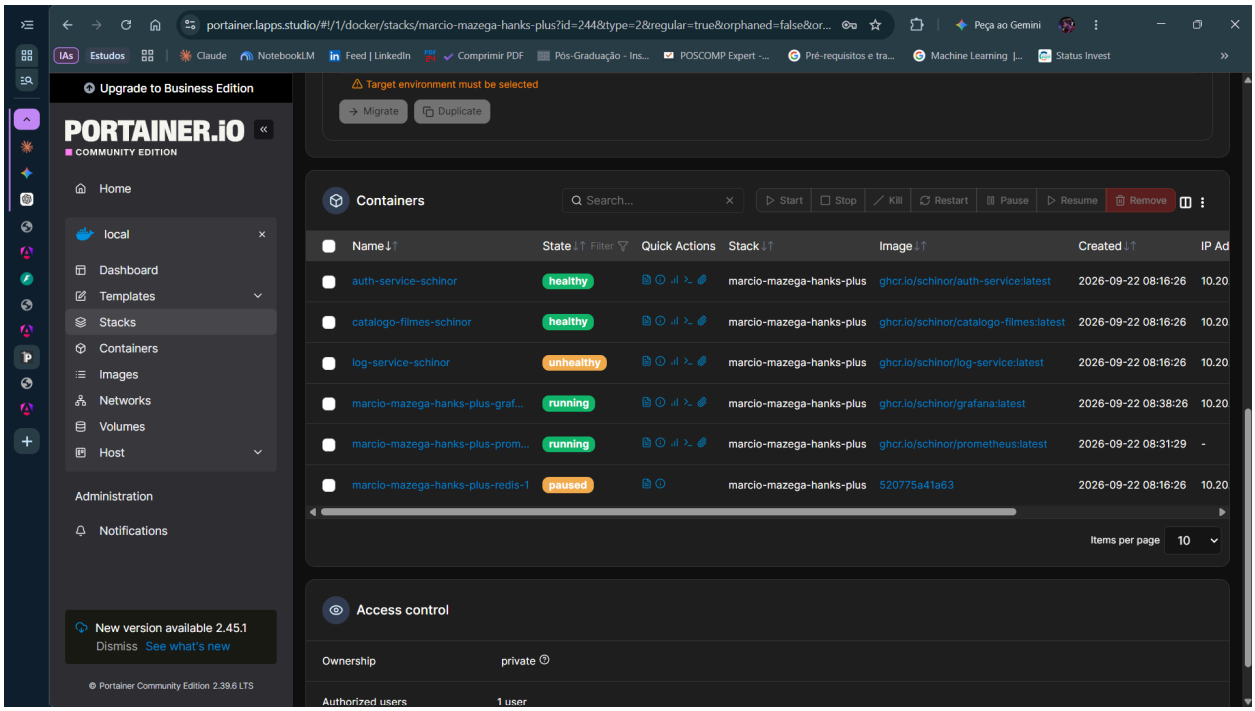


Figura 27: Atividade E3 — log-service unhealthy depois de pausar o Redis (imagem do README, adicionada no commit `ffad32e` em 22/09/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteLogsParado.png

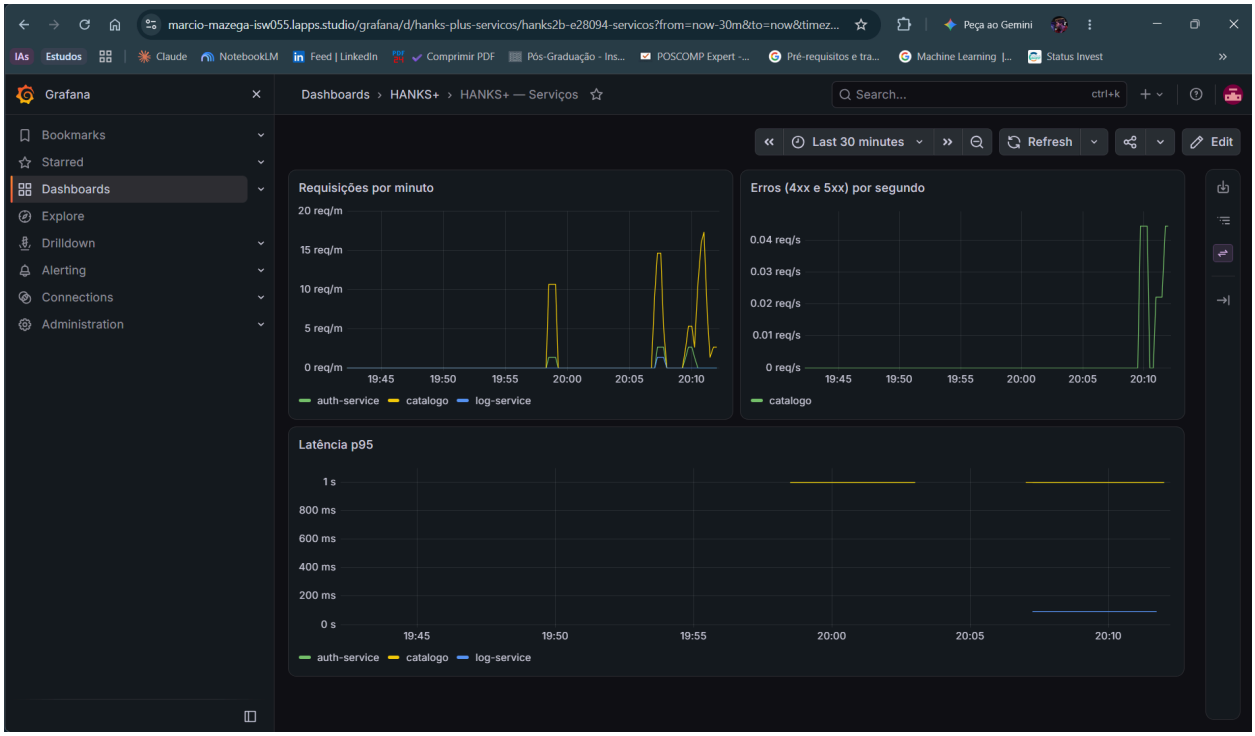


Figura 28: Atividade E3 — painel do Grafana com requisições por minuto, taxa de erro e latência p95 (imagem do README, adicionada no commit `1b0279d` em 02/10/2026). Imagem: https://github.com/Schinor/API-Filmes-Allan/blob/main/assets/SiteGrafanaDashboard.png

Dificuldades e como foram resolvidas. Tive problemas para relacionar as portas do Grafana com a porta acessada pela URL, por causa da forma como a infraestrutura converte o endereço público: o domínio passa por um proxy (Cloudflare) que só encaminha um conjunto fixo de portas, então as portas próprias do Grafana e do Prometheus nunca chegavam pela internet. Resolvi fazendo o catálogo expor `/grafana` e `/prometheus` e encaminhar internamente para os dois serviços (`6b5609b`, 22/09), e deixando essas portas

só na rede interna, porque publicá-las no servidor compartilhado conflitava com a stack de outro aluno (`c3af8c8`, 23/09). Depois ainda enfrentei problemas na comunicação interna do Grafana: os dados iam para o lugar errado e ele não conseguia encontrar os dados do sistema, então o login funcionava, mas os painéis ficavam sem dados. A causa era o proxy descartar o cabeçalho `Host`, o que fazia a proteção CSRF do Grafana recusar as consultas dos painéis; corrigi repassando os cabeçalhos corretamente (`1f800fd`, 28/09).

6 Considerações finais

Ao longo do bimestre, o autor aprendeu muito sobre testes de plataforma, cookies e segurança. Na infraestrutura, o trabalho foi principalmente de aperfeiçoar o que já sabia; os ganhos em segurança de plataforma, por outro lado, foram inigualáveis e funcionaram como a abertura de um novo mundo. Mesmo tratando-se do básico, a parte de segurança foi a mais surpreendente, presente em quase todas as entregas: permissões verificadas no backend (Atividade 4), registro das tentativas negadas (Atividade 5) e fotos servidas por URLs pré-assinadas com expiração (Atividade 6).

A parte de infraestrutura também ampliou os horizontes do autor, principalmente com o CI/CD, com o qual já tinha tido contato prévio. O Grafana e o Prometheus, por sua vez, foram surpresas, porque foi a primeira vez que o autor trabalhou com essas ferramentas.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Marcio Gabriel Schinor Mazega

Pompeia, 07/10/2026